Decoupling Support Enumeration and Value Discovery in Non-Binary ISD

Freja Elbro[1] and Paolo Santini[2]

[1] Danmarks Tekniske Universitet, Denmark
freel@dtu.dk
[2] Università Politecnica delle Marche, Italy
p.santini@univpm.it

Abstract. Information Set Decoding (ISD) refers to a class of algorithms designed to decode arbitrary linear codes over finite fields. It is among the state-of-the-art methods for solving the Syndrome Decoding Problem (SDP), which lies at the core of code-based cryptography. Since most cryptographic systems operate over the binary field, ISD algorithms are generally designed for this setting. However, emerging cryptographic systems, such as SDitH, that rely on the SDP over non-binary fields, make research into non-binary ISD increasingly relevant. While there have been numerous improvements to ISD algorithms for binary fields, generalizations of these improvements to non-binary fields have resulted in only modest runtime improvements. The only technique which applies specifically to non-binary fields was proposed only last year by Carrier, Hatey and Tillich and consists in solving the SDP over the projective space.
In this paper we continue along this line of research and introduce a novel technique to perform ISD in non-binary fields. Our key idea consists in speeding-up the enumeration of low weight vectors by enumerating only the supports and not the actual values, since these can be recovered by solving a (small) linear system. This idea is at the core of `LA-ISD`, the first algorithm we propose in the paper. We further enhance it by exploiting a meet-in-the-middle search leading to `MitM-LA`, the second algorithm we propose in this paper. We analyze our algorithms in both the finite and asymptotic regimes and show that they compare well with competitive solutions. In particular, `LA-ISD` results in the best memory-less algorithm, while `MitM-LA` is (slightly) faster than the state-of-the-art algorithm by Carrier, Hatey and Tillich for many parameter-sets as well as asymptotically.

Keywords: Code-based cryptography · Information Set Decoding · Non-binary Syndrome Decoding.

1 Introduction

The Syndrome Decoding Problem (SDP) is a well-known hard problem from coding theory, which plays an instrumental role in post-quantum cryptography.

Given a matrix $\mathbf{H} \in \mathbb{F}_q^{r \times n}$, with $\mathbb{F}_q$ the finite field with q elements, $\boldsymbol{s} \in \mathbb{F}_q^r$ and $w \in \mathbb{N}$, the SDP asks to find a weight-w vector $\boldsymbol{e} \in \mathbb{F}_q^n$ such that $\mathbf{H}\boldsymbol{e} = \boldsymbol{s}$. For our purposes, the weight refers to the Hamming weight. I.e. the number of non-zero entries.

For many parameter sets, the best solvers for SDP are known as Information Set Decoding (ISD) algorithms[3]. The core idea behind ISD is conceptually very simple and dates back to a 1962 paper by Prange [24]. In essence, Prange pointed out that it is sufficient to guess $\boldsymbol{e}$ on a so-called information set, as the remaining entries of $\boldsymbol{e}$ can be found via linear algebra. As the weight of $\boldsymbol{e}$ is small, Prange proposed to guess that $(\boldsymbol{e})_J$ ($\boldsymbol{e}$ restricted to the information set, J) was zero. If the guess was incorrect, his algorithm would simply choose another information set and continue like this until it found a set that did not intersect with the support of $\boldsymbol{e}$.

This foundational approach by Prange has since been the subject of extensive research, leading to numerous variants [1, 4, 9, 15, 17–20, 23, 25]. In all of these algorithms, Prange's original idea is modified by relying on trade-offs between the cost of each guess, and the average number of guesses needed. For instance, in the algorithm by Lee and Brickell [17], one allows $(\boldsymbol{e})_J$ to have a small weight. This makes guessing a successful information set more likely. But as the algorithm has to enumerate all possibilities for $(\boldsymbol{e})_J$, each guess of information set is more costly. The enumeration phase can be sped up by using a meet-in-the-middle approach, which leads to the algorithm introduced by Stern in 1988 [25]. Later techniques for improving ISD in the binary case employ so-called representations [1, 19], Nearest Neighbor searches [4, 20] or sieving approaches [9, 15].

ISD in non-binary fields Many of the improvements above have been successfully generalized to non-binary fields [22,23], where they have led to concrete and asymptotic speedups. However, Meurer showed in his thesis [22] that the benefit of the asymptotic speedups decreases as the field size increases. The main reason being that the cost of enumerating $(\boldsymbol{e})_J$ increases, as one has to enumerate not only the support of $(\boldsymbol{e})_J$, but also the value of $(\boldsymbol{e})_J$ on each non-zero position. And this becomes more costly as the field size increase.

In practice, when choosing parameters, the authors of SDitH rely on an algorithm from 2010 by Peters [23] which is a straight forward generalization of Stern's ISD to the case of non-binary fields (with some tweaks to save operations and reduce the cost of Gaussian Elimination). Likewise, the syndrome decoding estimator by Esser, Verbel, Zweydinger and Bellini [12] considers this to be the newest algorithm relevant for estimating the security of the q-ary syndrome decoding problems.

Hence, perhaps surprisingly, security analysis of syndrome decoding in the non-binary case relies to a large extend on an algorithm from 1988.

[3] For the binary finite field, dual attacks (also known as statistical decoding) outperform ISD for rates below 0.42 [6]

Leveraging the structure of non-binary finite fields This state-of-affairs has been questioned by a paper from Carrier et al., which appeared last year [7]. In the paper, the authors introduce an algorithm, which we will denote by `Proj-Stern`. This algorithm speeds up Stern's ISD by first reducing to the homogeneous case (that is, reducing SDP to the problem of finding a codeword with low weight) and then solving over the projective space. By working over the projective space, one can avoid enumeration of vectors that are equal up to multiplication by a scalar. Relying on some other techniques to save computations (e.g., pivots reusing as proposed in [5] and in [23]), Carrier et al. have shown that the SDitH parameters are actually slightly below the claimed security level.

To the best of our knowledge, the ideas in [7] constitute an unicum. Indeed, [7] targets directly the case of non-binary fields and, actually, leverages the field structure to speed-up existing algorithms. This leaves the question, whether there might exist other ideas for improving ISD in the non-binary case.

1.1 Our contribution

In this paper, we continue along this line of research and propose novel ISD algorithms for the non-binary case. We propose two algorithms, which we call `LA-ISD` and `MitM-LA`, where LA stands for linear algebra.[4] This name comes from the fact that, in both algorithms, the search for $(\boldsymbol{e})_J$ is made faster by leveraging some linear algebra observations. We exploit the fact that, when one searches for $(\boldsymbol{e})_J$, guessing its support is enough. Assuming $(\boldsymbol{e})_J$ is zero elsewhere, its coordinates can be found by solving a small linear system. Notice that the field size does not impact the complexity of this procedure.

The above procedure is, essentially, the description of how `LA-ISD` works. We further propose an enhanced version of this algorithm, which we call `MitM-LA`, in which the search for $(\boldsymbol{e})_J$ is performed in a meet-in-the-middle fashion. Due to some technicalities, we need to combine this phase with the representations technique (otherwise, the resulting algorithm would essentially collapse to existing ISD algorithms based on enumeration). We further enhance this algorithm with the idea from [7], namely, reducing to codeword finding and working over the projective space.

We consider parameters for schemes based on non-binary SDP from the literature [8,13,14,21]. We compare the costs of all relevant ISD algorithms, taking into account also tricks such as pivots reusing and factorization of information sets [5]. We additionally propose a trick to naturally reuse pivots without penalizing the success probability (as done in [5]). We show that, if one replaces the technique from [5] with our optimization in `Proj-Stern`, one can get a slight speed-up and an easier runtime analysis. Still, for some instances, this algorithm is slightly slower than `MitM-LA` which, is in many cases the fastest ISD.

[4] An open-source implementation of runtime analysis of `LA-ISD` and `MitM-LA` is available at: github.com/FrejaElbro/LAISD

We also analyze `MitM-LA` in the asymptotic regime and show that it is faster than `Proj-Stern`, but not as fast as the BJMM algorithm from [22], which remains the state-of-the-art for asymptotic improvements to non-binary ISD.

1.2 Organization of the paper

The notation we use throughout the paper, as well as necessary background notions about linear codes, are given in Section 2. A summary of the main ISD algorithms and tricks to speed up polynomial factors in their runtime are given in Section 3. In particular, this section contains also our observation on the natural reuse of pivots. The first algorithm we introduce in this paper, `LA-ISD`, is described and analyzed in Section 4. The second algorithm, `MitM-LA`, is instead presented in Section 5. Finally, in Section 6, we compare the various ISD algorithms.

2 Background

In this section we settle the notation we are going to use throughout the paper and recall basic concepts about linear codes.

As usual, for q a prime power, we use $\mathbb{F}_q$ to indicate a finite field with q elements. Matrices and vectors over $\mathbb{F}_q$ are written in bold, e.g., $\mathbf{A}$ is a matrix and $\boldsymbol{v}$ is a vector. For a matrix $\mathbf{A}$, the entry in the i-th row and j-th column is denoted by $a_{i,j}$, and similarly the i-th entry of a vector $\boldsymbol{v}$ is denoted by v_i. The identity matrix of size k is indicated as $\mathbf{I}_k$. A null matrix (or a null vector) is indicated as $\mathbf{0}$; its size will always be clear from the context. We use $GL_k(\mathbb{F}_q)$ to indicate the group of non singular, $k \times k$ matrices with entries over $\mathbb{F}_q$. The group of length-n permutations is indicated as S_n; with some abuse of notation, we see its elements as $n \times n$ matrices, called permutation matrices. For a set $J \subseteq \{1, \ldots, n\}$ of size m and a matrix $\mathbf{A} \in \mathbb{F}_q^{k \times n}$, $(\mathbf{A})_J$ denotes the $k \times m$ matrix formed by the m columns of $\mathbf{A}$ that are indexed by J. The complement of J is $J^C = \{1, \ldots, n\} \setminus J$. Analogously, for a vector $\boldsymbol{v} \in \mathbb{F}_q^n$, $(\boldsymbol{v})_J$ indicates the vector formed by the entries of $\boldsymbol{v}$ that are indexed by J. The support of a vector $\boldsymbol{v} \in \mathbb{F}_q^n$, denoted by $\mathrm{supp}(\boldsymbol{v})$, is the set of indices of non-zero coordinates, i.e., $\mathrm{supp}(\boldsymbol{v}) = \{i \in \{1, .., n\} \mid v_i \neq 0\}$. The size of the support is called the Hamming weight of $\boldsymbol{v}$ and is indicated as $\mathrm{wt}(\boldsymbol{v})$. We use $\mathcal{O}$ to denote the Landau symbol, which hides constant factors, and its variant $\tilde{\mathcal{O}}$, which also hides polylogarithmic factors. We define $\tilde{\mathcal{O}}$ by $f(n) \in \tilde{\mathcal{O}}(h(n)) \Leftrightarrow \exists k : f(n) \in \mathcal{O}(h(n) \log^k(h(n)))$. We will for example use $\tilde{\mathcal{O}}$ to estimate binomial coefficients through the well known approximation

$$\binom{n}{k} \in \tilde{\mathcal{O}}\left(2^{n \cdot h_2(k/n) + o(n)}\right), \tag{1}$$

where $h_2(x) = -x \log(x) - (1-x) \log(1-x)$ denotes the binary entropy function.

2.1 Linear codes

A linear code $\mathcal{C} \subseteq \mathbb{F}_q^n$ with dimension k and length n is a k-dimensional subspace of $\mathbb{F}_q^n$. The value $n-k$ is called code redundancy and $\kappa = k/n$ is called code rate. A code can be compactly represented via a generator matrix $\mathbf{G} \in \mathbb{F}_q^{k\times n}$, whose rows provides a basis for $\mathcal{C}$, or via a parity-check matrix $\mathbf{H} \in \mathbb{F}_q^{(n-k)\times n}$, which provides a basis for the null space of $\mathcal{C}$. Namely,

$$\mathcal{C} = \left\{ \mathbf{G}^\top \boldsymbol{u} \,\middle|\, \boldsymbol{u} \in \mathbb{F}_q^k \right\} = \left\{ \boldsymbol{c} \in \mathbb{F}_q^n \,|\, \mathbf{H}\boldsymbol{c} = \mathbf{0} \right\}.$$

For a vector, its product with the parity-check matrix is called its *syndrome*: the syndrome is zero only for *codewords*, i.e., for the vectors in $\mathcal{C}$.

By *information set*, we refer to a set $J \subseteq \{1,\dots,n\}$ of size k, such that for any two distinct codewords $\mathbf{c}, \mathbf{c}' \in \mathcal{C}$, it holds that $(\mathbf{c})_J \neq (\mathbf{c})'_J$. As a consequence, we have for any generator matrix $\mathbf{G}$ and any parity-check matrix $\mathbf{H}$ for $\mathcal{C}$, that $\mathbf{G}_J$ and $\mathbf{H}_{J^C}$ are non-singular.

It is clear that the same code admits multiple generator matrices. Indeed, for any $\mathbf{U} \in GL_k(\mathbb{F}_q)$, $\mathbf{G}$ and $\mathbf{UG}$ generate the same code.We say that a generator matrix is in *systematic form* if is in the form $\mathbf{G} = (\mathbf{I}_k, \mathbf{A})$, with $\mathbf{A} \in \mathbb{F}_q^{k\times(n-k)}$. Analogous reasoning applies to parity-check matrices. We say that a parity-check matrix is in systematic form if $\mathbf{H} = (\mathbf{A}, \mathbf{I}_{n-k})$, with $\mathbf{A} \in \mathbb{F}_q^{(n-k)\times k}$.

An important characteristic of a code is its *minimum distance*, which is defined as the minimum distance between any two codewords. For linear codes, the minimum distance corresponds to the minimum weight of non-zero codewords.

Random codes In many cryptographic applications, one deals with *random codes*, that is, codes that are sampled uniformly at random over the space of k-dimensional subspaces of $\mathbb{F}_q^n$. This is achieved by sampling, uniformly at random, the generator matrix or, equivalently, the parity-check matrix. This, in turn, is achieved by sampling only the non systematic part, for instance: we sample $\mathbf{A} \xleftarrow{\$} \mathbb{F}_q^{(n-k)\times k}$ and set $\mathbf{H} = (\mathbf{A}, \mathbf{I}_{n-k})$.[5]

For random codes, the minimum distance can be estimated with the well-known Gilbert Varshamov (GV) bound. This bound states that, for a random code with length n and dimension k, with large probability the minimum distance is

$$d_{\mathrm{GV}} = \max\left\{ w \in \{1,\dots,n\} \,\middle|\, \binom{n}{w}(q-1)^w < q^{n-k} \right\}. \tag{2}$$

Asymptotically, for $k = \kappa \cdot n$, where κ is constant, this turns into

$$d_{\mathrm{GV}} = \delta_{\mathrm{GV}} \cdot n \cdot \big(1 + o(1)\big), \quad \delta_{\mathrm{GV}} = h_q^{-1}(1-\kappa), \tag{3}$$

where $h_q(x) = \log_q(x) - x\log_q(x) - (1-x)\log_q(1-x)$ is the q-ary entropy function.

[5] To be precise, this allows to sample only the codes having $\{1,\cdots,k\}$ as information set. This issue can be circumvented by first sampling also a uniformly random permutation and then applying it to $(\mathbf{A}, \mathbf{I}_{n-k})$. This clarification has no impact on relevant quantities such as minimum distance and complexities of algorithms.

2.2 The Syndrome Decoding and Codeword Finding Problems

Formally, the search versions of SDP and CFP read as follows.

Problem 1 (Syndrome Decoding Problem (SDP)) *On input* $\mathbf{H} \in \mathbb{F}_q^{(n-k)\times n}$, $\mathbf{s} \in \mathbb{F}_q^{n-k}$ *and* $w \in \mathbb{N}$, *find* $\mathbf{e} \in \mathbb{F}_q^n$ *with Hamming weight* w *such that* $\mathbf{He} = \boldsymbol{s}$.

Problem 2 (Codeword Finding Problem (CFP)) *On input* $\mathbf{H} \in \mathbb{F}_q^{(n-k)\times n}$ *and* $w \in \mathbb{N}$, *find* $\mathbf{c} \in \mathbb{F}_q^n$ *with Hamming weight* w *such that* $\mathbf{Hc} = \mathbf{0}$.

In some sense, CFP can be thought of as the homogeneous version of SDP.

In cryptographic schemes, SDP is normally instantiated by sampling a random $\mathbf{H} \in \mathbb{F}_q^{(n-k)\times n}$ (possibly in systematic form) and a random $\boldsymbol{e} \in \mathbb{F}_q^n$ of weight w. One then sets $\boldsymbol{s} = \mathbf{H}\boldsymbol{e}$. For instance, in signature schemes such as [8, 13, 14], $\boldsymbol{e}$ is the secret key while the pair $\{\mathbf{H}, \boldsymbol{s}\}$ constitutes the public key.

In such a setting, existence of at least one solution is guaranteed. We call this regime "planted" (we inherit the terminology from the subset sum problem vocabulary) and an instance of this type is called planted SDP. One can also define a planted version of CFP, by first taking a generator matrix with $k-1$ rows chosen as random vectors and one row being a uniformly random weight-w vector, and then scrambling it with a random change of basis.

Notice that the average number of solutions to a planted SDP / CFP instance is given by

$$N_w = 1 + \frac{\binom{n}{w}(q-1)^w - 1}{q^{n-k}} \approx \max\left\{1\ ,\ \binom{n}{w}(q-1)^w q^{-(n-k)}\right\}.$$

Choosing w as the largest weight for which $N_w \leq 1$, we obtain the GV bound in (2) and (3).

When it comes to the cost of practical solvers, choosing $w \approx d_{\mathrm{GV}}$ maximizes the running time of all known solvers.

We describe state-of-the-art solvers for SDP and CFP in the next section.

3 Information Set Decoding in short

In this section, we first review the basic principles of ISD algorithms and provide a brief overview of the state-of-the-art methods. Next, we discuss techniques that can be used to accelerate the polynomial factors of ISD algorithm runtimes.

3.1 The main idea

All ISD algorithms are based on the following observation: to find a solution, $\boldsymbol{e}$, one just needs to determine its values on an information set.[6] Indeed, let

[6] This is the motivation behind the name "information set decoding".

$J \subseteq \{1, \ldots, n\}$ be an information set for $\mathcal{C}$ and assume $\boldsymbol{e}_J$ is known. Then, the missing part of $\boldsymbol{e}$ can be recovered as

$$(\mathbf{H})_J(\boldsymbol{e})_J + (\mathbf{H})_{J^C}(\boldsymbol{e})_{J^C} = \boldsymbol{s} \quad \Longrightarrow \quad (\boldsymbol{e})_{J^C} = (\mathbf{H})_{J^C}^{-1}\big(\boldsymbol{s} - (\mathbf{H})_J(\boldsymbol{e})_J\big). \quad (4)$$

Now, the problem becomes that of understanding the fastest way to guess $(\boldsymbol{e})_J$.

Current ISD algorithms are all based on the following simple approach: guess the weight of $(\boldsymbol{e})_J$ and enumerate all candidates.

For instance, in Prange's ISD algorithm, this weight is set to zero, so that no enumeration is required. The probability of choosing an information set J, such that $(\boldsymbol{e})_J$ has weight zero, is

$$P_{\text{perm}} = \frac{\binom{n-k}{w}}{\binom{n}{w}}.$$

Since there may be multiple solutions, we must consider this when calculating the expected number of guesses required to find a solution. We estimate the expected number of guesses needed to find at least one solution out of N_w to be

$$N_{\text{rounds}} = \max\left\{1, (N_w P_{\text{perm}})^{-1}\right\}, \quad (5)$$

Prange's algorithms repeatedly samples information sets, computes $(\boldsymbol{e})_{J^C}$ using (4) and checks if it has the desired weight; if this is not the case, the algorithm restarts with another information set. Taking $T_{\text{Gauss}} = (n-k)^2(n+k)$ as the cost to solve (4), one gets an algorithm using, on average, a number of $\mathbb{F}_q$-operations given by

$$T_{\texttt{Prange}} = \mathcal{O}(N_{\text{rounds}} T_{\text{Gauss}}).$$

3.2 Advanced ISD algorithms

The approach we just described can be improved by trading off the cost of each iteration with the success probability of each iteration. In the literature, one finds many ISD variants [1, 3, 9, 10, 19, 20] which are all based on the following approach:

1. Choose a set $J \subset \{1, \ldots, n\}$ of size $k + \ell$, for some non-negative integer $\ell \leq n - k$, which will be chosen to minimise runtime. Note that, for random codes, such a set contains an information set with high probability.
2. Set $\mathbf{P} \in S_n$ so that the coordinates indexed by J are moved to the leftmost $k + \ell$ coordinates and, with (Partial) Gaussian Elimination compute $\mathbf{U} \in GL_{n-k}(\mathbb{F}_q)$ such that

$$\mathbf{UHP} = \begin{pmatrix} \mathbf{H}' & \mathbf{0} \\ \mathbf{H}'' & \mathbf{I}_{n-k-\ell} \end{pmatrix},$$

with $\mathbf{H}'$ and $\mathbf{H}''$ having sizes $\ell \times (k+\ell)$ and $(n-k-\ell) \times (k+\ell)$, respectively. The syndrome $\boldsymbol{s}$ and the solution $\boldsymbol{e}$ get transformed accordingly:

$$\boldsymbol{s} \quad \longrightarrow \mathbf{U}\boldsymbol{s} = \begin{pmatrix} \boldsymbol{s}' \\ \boldsymbol{s}'' \end{pmatrix},$$

$$\boldsymbol{e} \quad \longrightarrow \mathbf{P}^{-1}\boldsymbol{e} = \begin{pmatrix} \boldsymbol{e}' \\ \boldsymbol{e}'' \end{pmatrix}.$$

Thus, we get two equations of the form:

$$\begin{cases} \mathbf{H}'\boldsymbol{e}' = \boldsymbol{s}', \\ \boldsymbol{e}'' = \boldsymbol{s}'' - \mathbf{H}''\boldsymbol{e}'. \end{cases}$$

3. Solve the smaller instance given by $\mathbf{H}', \boldsymbol{s}'$ and target weight w', getting a list $\mathscr{L}$ containing candidates for $\boldsymbol{e}'$. w' is again a parameter of the algorithm chosen to minimise runtime.
4. For all $\boldsymbol{e}' \in \mathscr{L}$, compute $\boldsymbol{e}'' = \boldsymbol{s}'' - \mathbf{H}''\boldsymbol{e}'$.
5. If $\mathrm{wt}(\boldsymbol{e}'') = w - w'$ return the solution $\boldsymbol{e} = \mathbf{P}\begin{pmatrix} \boldsymbol{e}' \\ \boldsymbol{e}'' \end{pmatrix}$. Else, return to step 1 and choose a different set J.

Most of the ISD algorithms only differ in step 3. i.e., how to find the solution to the smaller instance.

The Lee-Brickell algorithm Lee and Brickell [17] were the first to introduce the possibility of errors in $\boldsymbol{e}'$. Their idea to solve the smaller instance was simple: Enumerate all possibilities for $\boldsymbol{e}'$ of weight w'.

In `LeeBrickell`, ℓ is set to zero. Hence, for a field size of q, there are

$$L = \binom{k}{w'}(q-1)^{w'}$$

possibilities for $\boldsymbol{e}'$ of weight w'. The probability of finding a specific solution $\boldsymbol{e}$ in one round of the algorithm is equal to the probability that $\mathbf{P}$ permutes the entries of $\boldsymbol{e}$ such that exactly w' non-zero entries end up on the first k coordinates. This probability is

$$P_{perm} = \frac{\binom{k}{w'}\binom{n-k}{w-w'}}{\binom{n}{w}}.$$

And the runtime of `LeeBrickell` is thus in

$$\mathcal{O}\big((N_w P_{\mathrm{perm}})^{-1} \cdot (T_{\mathrm{Gauss}} + T_{\mathrm{check}} \cdot L)\big),$$

where T_{check} is the time it takes to check the weight of $\boldsymbol{e}''$. We will get back to how this can be estimated later.

Stern and Dumer's Algorithms Stern [25] offered the first asymptotic speed-up of Prange's algorithm. His algorithm was slightly sped-up by Dumer [10] and generalized to arbitrary fields by Peters [23]. We describe here the generalized algorithm by Dumer.

In Dumer's algorithm, the subinstance (step 3) is solved via a collision search. We are given $\mathbf{H}' \in \mathbb{F}_q^{\ell \times (k+\ell)}$, $\boldsymbol{s}' \in \mathbb{F}_q^{\ell}$ and w'. For simplicity, let us assume that $k+\ell$ and w' are even. Dumer's algorithm splits the sought-after vector as $\boldsymbol{e}' = \boldsymbol{e}_1 + \boldsymbol{e}_2$, where $\boldsymbol{e}_1 \in \{0\}^{(k+\ell)/2} \times \mathbb{F}_q^{(k+\ell)/2}$ and $\boldsymbol{e}_2 \in \mathbb{F}_q^{(k+\ell)/2} \times \{0\}^{(k+\ell)/2}$ are both assumed to have weight $w'/2$. Thus, the syndrome equation of the subinstance becomes

$$\mathbf{H}'\boldsymbol{e}_1 + \mathbf{H}'\boldsymbol{e}_2 = \boldsymbol{s}'.$$

and the algorithm goes as follows

1. Build a list of candidates $\boldsymbol{e}_1$,

$$\mathscr{L}_1 = \{(\boldsymbol{e}_1, \mathbf{H}'\boldsymbol{e}_1) \mid \boldsymbol{e}_1 \in \{0\}^{(k+\ell)/2} \times \mathbb{F}_q^{(k+\ell)/2}, \mathrm{wt}(\boldsymbol{e}_1) = w'/2\}$$

   and similarly a list for $\boldsymbol{e}_2$:

$$\mathscr{L}_2 = \{(\boldsymbol{e}_2, \boldsymbol{s}' - \mathbf{H}'\boldsymbol{e}_2) \mid \boldsymbol{e}_2 \in \mathbb{F}_q^{(k+\ell)/2} \times \{0\}^{(k+\ell)/2}, \mathrm{wt}(\boldsymbol{e}_2) = w'/2\}.$$

2. Go through all elements in $\mathscr{L}_1 \times \mathscr{L}_2$ and search for a collision, i.e., a pair $\big((\boldsymbol{e}_1, \boldsymbol{a}), (\boldsymbol{e}_2, \boldsymbol{a})\big) \in \mathscr{L}_1 \times \mathscr{L}_2$. For each collision add the candidate $\boldsymbol{e}' = \boldsymbol{e}_1 + \boldsymbol{e}_2$ to the list $\mathscr{L}$.

The probability of finding a specific solution $\boldsymbol{e}$ in one round of the algorithm is the probability that $\mathbf{P}$ permutes the entries of $\boldsymbol{e}$ in such a way that exactly $w'/2$ non-zero entries land on the first $(k+\ell)/2$ coordinates and $w'/2$ non-zero entries land on the next $(k+\ell)/2$ coordinates. This probability is

$$P_{\mathrm{perm}} = \frac{\binom{(k+\ell)/2}{w'/2}^2 \binom{n-k-\ell}{w-w'}}{\binom{n}{w}}.$$

The cost of each round is

$$T_{\mathrm{Gauss}} + T_{\mathrm{lists}} + T_{\mathrm{check}} \cdot N_{\mathrm{cols}},$$

where T_{lists} is the time it takes to build the lists $\mathscr{L}_1$ and $\mathscr{L}_2$, N_{cols} is the expected number of collisions and T_{check} is the time it takes to check the weight of $\boldsymbol{e}''$ for each collision. We will go into details with each of these when we discuss concrete algorithms.

The BJMM algorithm A more sophisticated idea is to use the representation technique introduced for subsetsums in [16]. The idea was first used for decoding over the binary field in [19] and has later been improved in [1] and extended to arbitrary finite fields in [22].

The idea is to write $\boldsymbol{e}' = \boldsymbol{e}_1 + \boldsymbol{e}_2$ with $\boldsymbol{e}_1, \boldsymbol{e}_2 \in \mathbb{F}_q^{k+\ell}$ both of weight $w_1 = w'/2 + \varepsilon$. The two vectors $\boldsymbol{e}_1$ and $\boldsymbol{e}_2$ overlap in ε many positions, in such a way, that when adding them together, these positions cancel out. We hence enlarge the size of the base lists. Additionally, we get many representations $(\boldsymbol{e}_1, \boldsymbol{e}_2)$ leading to the same $\boldsymbol{e}' = \boldsymbol{e}_1 + \boldsymbol{e}_2$. Let R denote the number of ways to write $\boldsymbol{e}'$ of weight w' as a sum of vectors $\boldsymbol{e}_1, \boldsymbol{e}_2$ of weight w_1.

If we go through all possible $\boldsymbol{e}_1, \boldsymbol{e}_2$ and add $\boldsymbol{e}' = \boldsymbol{e}_1 + \boldsymbol{e}_2$ to the list $\mathscr{L}$, then every candidate $\boldsymbol{e}' \in \mathbb{F}_q^{k+\ell}$ of weight w' is represented R times in that list. We, thus, do not have to go through all possible $\boldsymbol{e}_1, \boldsymbol{e}_2$. By requiring the vectors $\boldsymbol{e}_1, \boldsymbol{e}_2$ to add up to the desired syndrome on $u \leq \ell$ many positions, we can set $u = \log_q(R)$ and expect that at on average, one representation of $\boldsymbol{e}'$ is constructed.

The algorithm in [22] uses two levels, that is, we split the vectors $\boldsymbol{e}_1, \boldsymbol{e}_2$ one more time, as $\boldsymbol{e}_1 = (\mathbf{x}_1, \mathbf{y}_1)$ and $\boldsymbol{e}_2 = (\mathbf{x}_2, \mathbf{y}_2)$, where $\mathbf{x}_i, \mathbf{y}_i \in \mathbb{F}_q^{(k+\ell)/2}$ for $i \in \{1, 2\}$ and all vectors have weight $w_1/2$. The reader is encouraged to consider how the algorithm generalises to more levels, or to look this up in the original paper [1].

The algorithm with two level starts by creating four lists containing all possibilities for $\boldsymbol{x}_1$, $\boldsymbol{y}_1$, $\boldsymbol{x}_2$ and $\boldsymbol{y}_2$ respectively. These lists all have size

$$B = \binom{(k+\ell)/2}{w_1/2}.$$

Similarly to Dumers algorithm, these lists are then combined two and two through a collision search to form two lists, $\mathscr{L}_1, \mathscr{L}_2$ which contain all possibilities for $\boldsymbol{e}_1$ and $\boldsymbol{e}_2$ respectively. These lists have sizes

$$L = B^2/q^u = B^2/R.$$

Finally, $\mathscr{L}_1$ and $\mathscr{L}_2$ are again combined through a collision search on the remaining $\ell - u$ syndrome equations. The expected number of collisions is

$$C = L^2/q^{\ell-u}.$$

For each collision, it is checked if the vector has weight w', and if it does, then it is added to the list $\mathscr{L}$.

The cost of solving the small instance is thus

$$B + L + C$$

up to subexponential factors. Similarly, up to subexponential factors, the expected number of rounds before a successful permutation is found is

$$\frac{\binom{n}{w}}{\binom{k+\ell}{w'}\binom{n-k-\ell}{w-w'}}.$$

3.3 Which ISD algorithms to consider

Settling which ISD algorithms to consider for finite regime analysis is difficult. From asymptotic analysis, we know that `BJMM` is faster than `Stern` and `Dumer` for all finite fields tested so far [22]. Also the security analysis of Classic McEliece shows ISD's with multiple layers like `BJMM` constitute the fastest attacks [11]. However, for non-binary schemes the cost of enumerating all vectors of a certain weight increases with the field size, as one has to not only enumerate the positions, but also the values. In fact, Meurer [22] shows that the weight of the vectors in the base lists go to zero for growing q and in [7] they find $w' = 2$ or 4 to be optimal for `Proj-Stern` and SDitH parameters. For such small vectors, we believe the benefits of multiple level ISD's to be minimal.

Furthermore the advanced algorithms come with an increasingly heavy cost in memory. In fact [22] found that with an upper limit on memory of 2^{40}, Prange's algorithm performs better asymptotically than `Stern` and `BJMM` , as no memory is allocated at all.

For these reasons we follow convention set in [12, 21], and do not consider algorithms with more levels than `Stern` when comparing costs of ISD algorithms on q-ary syndrome decoding problems.

3.4 Ways to polynomially speed-up ISD

When parameters are big enough, one can usually ignore polynomial factors when estimating runtime, as they contribute very little compared to the exponential sizes of the lists. However, for smaller parameters, they become relevant. Therefore, the literature is rich with big and small tricks to minimize the polynomial factors of the ISD runtime. In this section we present the ones we will use, and the ones used the projective Stern algorithm in [7], which we consider to be state-of-the-art for syndrome decoding for non-binary fields and parameters of cryptographic interest. We finish the section by presenting a new approach to minimizing the cost of Gaussian Elimination, which is based on the same idea as the one in [5], but which is simpler to analyze.

Using the projective structure of $\mathbf{He} = \mathbf{0}$ In [7], it is shown how to speed-up ISD for finite regimes by utilizing the following projective structure: if $\boldsymbol{s} = \mathbf{0}$, and $\boldsymbol{e}$ solves the syndrome decoding problem, then $\alpha\boldsymbol{e}$ also solves the syndrome decoding problem for all $\alpha \neq 0$. To solve the syndrome decoding problem, it suffices to find just one scalar multiple of $\boldsymbol{e}$. The idea in [7] is to construct a variant of Stern's algorithm, where only one of these scalar multiples is enumerated. This has the potential to decrease total runtime by a factor of q.

For the case where $\boldsymbol{s} \neq \mathbf{0}$, [7] proves that the standard reduction from syndrome decoding problem to codeword finding problem does not increase runtime significantly. More specifically,

Theorem 1. *Let a code $\mathcal{C}$ be the right kernel of a parity-check matrix $\mathbf{H} \in \mathbb{F}_q^{(n-k)\times n}$ and let $\boldsymbol{s} \in \{\mathbf{H}\tilde{\boldsymbol{e}} \mid \mathrm{wt}(\tilde{\boldsymbol{e}}) = t\}$ for $t \in [0; n]$. Let $\mathcal{C}' \triangleq \langle \mathcal{C}, \boldsymbol{z} \rangle$ be the code*

generated by the codewords in $\mathcal{C}$ and any word $\boldsymbol{z} \in \mathbb{F}_q^n$ such that $\mathbf{H}\boldsymbol{z} = \boldsymbol{s}$. We denote by $\mathbf{H}'$ a parity check matrix of this augmented code. Then, on average over $\mathbf{H}$, a solution to the syndrome decoding problem $SDP(\mathbf{H}, \boldsymbol{s}, t)$ can be found by solving the codeword finding problem $CFP(\mathbf{H}', t)$ $q/(q-1)$ times.

Reusing pivots in the Gaussian Elimination In [5], Canteaut and Chabaud propose reducing the cost of Gaussian elimination by modifying information sets incrementally – swapping only one coordinate at a time. This idea is generalized in [2], which introduces a parameter c measuring how many coordinates of the information set are replaced in each round. On the positive side, [7] estimate that this reduces the cost of a full Gaussian elimination to

$$T_{Gauss} = c(n-k)(2k+c+1).$$

On the negative side, the probability of success in a round is no longer independent of the probability of success in the previous round. As a result, estimating the expected runtime requires more advanced techniques – such as the Markov chain analysis used in [5].

Natural Reuse of Pivots To get around the Markov Chain analysis of the previous section, we make the simple observation that with high probability, pivots can be reused even when information sets are chosen at random. We are thus able to reduce the expected cost of Gaussian elimination without penalizing the success probability of each iteration. Note that this observation was already made in [2]. For the sake of completeness, we briefly recall it here.

Without loss of generality, we can assume that the input $\mathbf{H}$ is in systematic form. Indeed, this is guaranteed in many cases; even if this is not true, one can put it in systematic form before calling ISD.

Let $J \subseteq \{1, \ldots, n\}$ denote the information set which we sample, uniformly at random, at the beginning of each iteration, and $J^C = \{1, \ldots, n\} \setminus J$. Let u denote the average size of $J^C \cap \{k+1, \ldots, n\}$: in other words, out of the $n-k$ columns we need to pivot, there are on average u columns which are already pivoted.

With a simple reordering of columns and rows reordering, we obtain a matrix whose last u columns already form part of an identity matrix, hence we can start pivoting from the column in position $n-u$ (we are assuming columns are numbered in ascending order, going from left to right). See Figure 1 for a representation of the procedure.

It can be seen (details are given in Appendix B) that $u = (n-k)^2/n$ and that this observation leads to the following cost for Gaussian elimination

$$\begin{aligned} T_{\text{Gauss}} &= \sum_{i=1}^{n-k-u} k+i-1+2(n-k-1)(k+i-1) \\ &= (n-k-\tfrac{1}{2})(n-k-u)(n+k-u-1). \end{aligned} \tag{6}$$

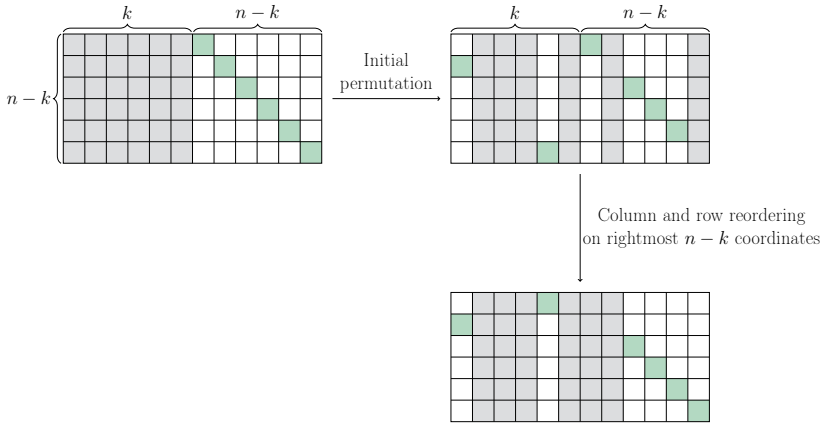


Fig. 1: Visualization of how pivots can be reused, when one starts from a matrix in systematic form. In this case, $n = 12$ and $k = r = 6$. The chosen information set is such that 4 of the 6 columns on the rightmost part are already pivoted; hence, columns and row reordering places the 4×4 identity in the rightmost, bottom corner.

4 LA-ISD: the basic idea

In this section, we describe the main idea underlying our new ISD algorithm and analyze its time complexity. In the next section, we show how the algorithm can be sped-up using techniques from other ISD algorithms, such as the meet-in-the-middle approach and working over the projective space.

4.1 Using Linear Algebra to solve the small instance

We first give the main intuition behind the algorithm, then provide full algorithmic details and analyze the resulting time complexity.

LA-ISD follows the generic ISD approach from Section 3.2. After an initial permutation and partial Gaussian elimination, the problem is reduced to finding a low-weight solution $\boldsymbol{e}'$ to a smaller system $\mathbf{H}'\boldsymbol{e}' = \boldsymbol{s}'$. LA-ISD focuses on finding a solution $\boldsymbol{e}'$ of weight $w' = \ell$. It achieves this by iterating through all possible supports $S \subseteq \{1, ..., k + \ell\}$ of size ℓ. For each chosen support S, the algorithm attempts to solve the linear system $(\mathbf{H}')_S(\boldsymbol{e}')_S = \boldsymbol{s}'$ to find the entries of a potential candidate $\boldsymbol{e}'$. If the submatrix $(\mathbf{H}')_S$ is non-singular, a unique candidate is found.[7] Once a candidate is found, the algorithm checks if it leads to a valid solution for the original problem by computing the full error vector and verifying its weight.

Full details on how the algorithm is implemented are given in Algorithm 1.

[7] For simplicity, if the matrix is singular, our algorithm skips the corresponding support S. This event occurs with low probability (approximately $1/q + 1/q^2$) for a random $\mathbf{H}'$, and its impact on the overall performance is therefore small.

Algorithm 1: LA-ISD

Data: parameter $\ell \in \mathbb{N}$, $0 \leq \ell \leq n-k$
Input: Matrix $\mathbf{H} \in \mathbb{F}_q^{r \times n}$, syndrome $\boldsymbol{s} \in \mathbb{F}_q^r$, weight $w \in \mathbb{N}$
Output: vector $\mathbf{e}' \in \mathbb{F}_q^n$ with weight w, such that $\mathbf{H}\boldsymbol{e} = \boldsymbol{s}$

while True **do**
  /* Permute and bring into row reduced echelon form */
  Sample $\mathbf{P} \xleftarrow{\$} S_n$;
  If it exists, compute $\mathbf{U} \in GL_r(\mathbb{F}_q)$ so that $\mathbf{UHP} = \begin{pmatrix} \mathbf{H}' & \mathbf{0} \\ \mathbf{H}'' & \mathbf{I}_{n-k-\ell} \end{pmatrix}$; otherwise, go back to instruction 2;
  Set $\mathbf{U}\boldsymbol{s} = \begin{pmatrix} \boldsymbol{s}' \\ \boldsymbol{s}'' \end{pmatrix}$; // $\boldsymbol{s}' \in \mathbb{F}_q^{\ell}$, $\boldsymbol{s}'' \in \mathbb{F}_q^{n-k-\ell}$
  /* Loop over all supports for $\boldsymbol{e}'$ */
  for $S \subseteq \{1, \ldots, k+\ell\}, \quad |S| = \ell$ **do**
    /* Continue only if the matrix $\mathbf{H}'_S$ is non singular */
    if $\mathrm{rank}(\mathbf{H}')_S = \ell$ **then**
      Initialize $\boldsymbol{e}' = \mathbf{0}$; // Initialized as the null vector
      Set $\boldsymbol{e}'_S = \mathbf{H}'^{-1}_S \boldsymbol{s}'$; // Set entries in support S
      if $\mathrm{wt}(\boldsymbol{e}') = \ell$ **then**
        /* Compute $\boldsymbol{e}''$ and check resulting weight */
        Compute $\boldsymbol{e}'' = \boldsymbol{s}'' - \mathbf{H}''\boldsymbol{e}'$;
        if $\mathrm{wt}(\boldsymbol{e}'') = w - \ell$ **then**
          return $\mathbf{P}\begin{pmatrix} \boldsymbol{e}' \\ \boldsymbol{e}'' \end{pmatrix}$;

4.2 Complexity analysis

For the sake of simplicity, we use the estimate in Eq. (6) for the cost of (partial) Gaussian elimination, even though the real cost is actually slightly smaller since we only need pivots in $n-k-\ell$ columns (instead of $n-k$).

Proposition 1 (Time complexity of LA-ISD).
Let $\mathbf{H} \xleftarrow{\$} \mathbb{F}_q^{n-k \times n}$ *and* $\boldsymbol{e} \in \mathbb{F}_q^n$ *be a uniformly random vector with weight* w*. Let* $\mathbf{s} = \mathbf{H}\boldsymbol{e}$*. Then, Algorithm 1, on input* $\{\mathbf{H}, \boldsymbol{s}, w\}$*, halts on average in runtime*

$$\mathcal{O}\Big(N_{\mathrm{rounds}} \cdot \big(T_{\mathrm{check}} + T_{\mathrm{Gauss}} + T_{\mathrm{Sys}}\big) \cdot \log_2(q)\Big),$$

where $T_{\mathrm{Sys}} = \ell^2(\ell+1)\binom{k+\ell}{\ell}$, T_{Gauss} is as in Eq. (6) and $T_{\mathrm{check}} = N_{\mathrm{small}} \cdot \frac{q}{q-1}(w-\ell+1)\ell\left(1+\frac{q-2}{q-1}\right)$ with

$$N_{\mathrm{small}} = \left(1 - \frac{1}{q} - \frac{1}{q^2}\right)\left(1 - \frac{1}{q}\right)^{\ell}\binom{k+\ell}{\ell},$$

and $N_{rounds} = \max\left\{1, (P_{perm} \cdot N_w)^{-1}\right\}$ with

$$P_{perm} = \frac{\binom{k+\ell}{\ell}\binom{n-k-\ell}{w-\ell}}{\binom{n}{w} \cdot \left(1 - \frac{1}{q} - \frac{1}{q^2}\right)}.$$

Proof. First, the algorithm has negligible memory cost since it only requires to store the input matrix and syndrome. We consider the variant of the algorithm with the natural pivots re-usage. The cost for the Gaussian elimination step can therefore be taken equal to Eq. (6).

Let us now consider the number of valid vectors which are found by the algorithm in each loop. For each set S, we solve the linear system in runtime $(\ell-1)^2(\ell+1)$. Indeed, this can be done by preparing a matrix $\left(\boldsymbol{s}', (\mathbf{H}')_S\right)$, which is a $\ell \times (\ell+1)$ matrix, and then bringing it into row reduced echelon form. We repeat this for all subsets S, of which there are $\binom{k+\ell}{\ell}$. So, the cost to find all candidates for $\boldsymbol{e}'$ can be taken as $T_{\text{Sys}} = \ell^2(\ell+1)\binom{k+\ell}{\ell}$.

Now, we derive the number of candidates for which we actually compute also $\boldsymbol{e}''$. First, for each set S, the probability the system $\mathbf{H}'_S$ is non singular, is approximately $\left(1 - \frac{1}{q} - \frac{1}{q^2}\right)$. The probability that the solution $\boldsymbol{e}'$ has full weight is given by $\left(1 - \frac{1}{q}\right)^{\ell}$. Hence, the number of candidates we need to test is

$$N_{\text{small}} = \left(1 - \frac{1}{q} - \frac{1}{q^2}\right)\left(1 - \frac{1}{q}\right)^{\ell}\binom{k+\ell}{\ell}.$$

The average cost of checking whether a candidate extends to a solution of the original problem is

$$\frac{q}{q-1}(w-\ell+1)\ell\left(1 + \frac{q-2}{q-1}\right),$$

where we make use of the early-abort technique from [23, Section 4: Collisions].[8] Then, the cost to check all candidates for $\boldsymbol{e}'$ can be taken as

$$T_{\text{check}} = N_{\text{small}} \cdot \frac{q}{q-1}(w-\ell+1)\ell\left(1 + \frac{q-2}{q-1}\right).$$

We conclude the proof by considering that, for a single solution, the probability that it is found in one iteration of our algorithm is

$$P_{\text{perm}} = \frac{\binom{k+\ell}{\ell}\binom{n-k-\ell}{w-\ell}}{\binom{n}{w} \cdot \left(1 - \frac{1}{q} - \frac{1}{q^2}\right)},$$

where $\left(1 - \frac{1}{q} - \frac{1}{q^2}\right)$ accounts for the fact that, even if the permutation is valid, it may happen that the corresponding subset S does not lead to a submatrix $(\mathbf{H}')_S$ which has full rank. Finally, we rely on Eq. (5) to estimate the average number of iterations. □

[8] This technique computes $\boldsymbol{e}''$ entry by entry, exploiting the fact that multiplications by 1 are free, and aborts as soon as the weight of the calculated vector exceeds $w-\ell$.

Remark 1. Following the ideas in [2,5], one could modify the algorithm so that, in each iteration, the permutation leaves a fixed (and large) subset of the leftmost $k+\ell$ coordinates unchanged. This would simplify the Gaussian elimination step by reducing the number of columns that need to be updated. However, we chose not to adopt this strategy in our analysis for three main reasons. First, enforcing such a constraint would significantly complicate the analysis. Second, as we argue in the next section, our algorithm already implicitly reuses some pivot positions in a way that resembles the approach of [5]. Third, we conducted preliminary experiments with small values of c, the number of pivots changed across iterations, and observed little to no improvement in performance.

4.3 Comparison to other ISD algorithms

On a high level, `LA-ISD` can be seen as variant of Prange's algorithm. To find $\boldsymbol{e}$, `LA-ISD` needs to identify a permutation $\mathbf{P}$ and a set S such that $(\boldsymbol{e}')_{S^C} = \mathbf{0}$. In other words, like Prange's algorithm, it has to find k zero positions of $\boldsymbol{e}$. Once these positions are found, the remaining entries of $\boldsymbol{e}$ are determined using linear algebra. [9]

The key difference between `LA-ISD` and Prange's algorithm lies in how we search for the zero positions. In Prange's algorithm, we randomly sample information sets, and if one information set does not work, we select a new one at random. For each information set, we perform Full Gaussian Elimination. In contrast, `LA-ISD` selects a set of size $k+\ell$ at random, performs Partial Gaussian Elimination, and then tests all possible subsets of size k before selecting a new set of size $k+\ell$. This approach allows `LA-ISD` to reduce the number of Gaussian Elimination steps, as testing each of the $\binom{k+\ell}{k}$ possible subsets is relatively cheap (since we only need to solve a linear system with ℓ equations and ℓ unknowns). Only after checking all of these sets does `LA-ISD` perform a new round of Gaussian Elimination.

In this way, the idea behind `LA-ISD` is somewhat similar to the pivot-reusing technique introduced by [5]. Both methods reduce the cost of Gaussian Elimination by recognizing that it is not necessary to discard the entire information set after each round. Still, the main difference between `LA-ISD` and pivot-reusing lies in how the information set is updated. Indeed, the method used in [5] enforces that each new iteration uses an information set in which only a fixed amount of coordinates is different from the coordinates used in the previous iteration. This holds for all iterations, so that every two subsequent iterations are dependent.

In contrast, `LA-ISD` works such that only $\binom{k+\ell}{\ell}$ consecutive iterations are dependent. Indeed, in each iteration, the algorithm tests all the size k-sets which are subsets of a set of size $k+\ell$: so, every two distinct size-k sets have at most ℓ different coordinates. However, differently from [5], after $\binom{k+\ell}{\ell}$ sets are checked, the algorithm starts from scratch by selecting an independent subset of size $k+\ell$, which makes the runtime analysis simpler.

[9] However, differently from Prange's algorithm, it also requires to determine ℓ positions where $\boldsymbol{e}$ is nonzero.

5 Meet-in-the-middle LA-ISD (MitM-LA)

In this section, we consider the possibility of using a meet-in-the-middle approach to solve the small instance, similarly to Dumer's ISD. However, we have to face an obstacle here, because we cannot split $\boldsymbol{e}' = \boldsymbol{e}_1 + \boldsymbol{e}_2$ as Dumer's ISD does. Indeed, to find these vectors in the same way as LA-ISD, we would need to know their syndromes $\boldsymbol{s}_1 = \mathbf{H}'\boldsymbol{e}_1$ and $\boldsymbol{s}_2 = \mathbf{H}'\boldsymbol{e}_2$. However, these vectors are unknown. We only know that they sum to $\boldsymbol{s}'$.

Guessing the pair $(\boldsymbol{s}_1, \boldsymbol{s}_2)$ would make little sense because each guess is valid with probability $q^{-\ell}$. Hence, we would need to solve the small instance approximately, q^ℓ times. To avoid this overhead, we propose to rely on *representations*: instead of splitting $\boldsymbol{e}'$ into two vectors with disjoint supports, we allow $\boldsymbol{e}_1$ and $\boldsymbol{e}_2$ supports in all $k+\ell$ coordinates. Since there are now multiple pairs $(\boldsymbol{e}_1, \boldsymbol{e}_2)$ that sum to the same vector, we increase the probability that one choice for $(\boldsymbol{s}_1, \boldsymbol{s}_2)$ is valid.

This (with some more technicalities) constitutes the core idea of MitM-LA. We proceed by introducing the algorithm and providing its runtime analysis. We then proceed to propose another improvement based on exploiting the projective space, as introduced in [7].

5.1 The algorithm

The description of MitM-LA is given in Algorithm 2.

Our algorithm depends on three integers, $0 \leq \ell \leq n-k$, $0 \leq w' \leq k+\ell$ and $w'/2 \leq w_0 \leq \ell$ which are optimized to minimize runtime. We follow the framework of section 3.2, and start by sampling uniformly at random a permutation $\mathbf{P}$ and by bringing $\mathbf{HP}$ in partial systematic form (lines 2–4). Then, we split $\mathbf{H}' = \begin{pmatrix}\mathbf{H}_1\\ \mathbf{H}_2\end{pmatrix}$, with $\mathbf{H}_1$ a $w_0 \times (k+\ell)$ matrix and $\mathbf{H}_2$ a $(\ell - w_0) \times (k+\ell)$ matrix. Analogously, we split the syndrome $\boldsymbol{s}'$ as $\begin{pmatrix}\boldsymbol{s}_1\\ \boldsymbol{s}_2\end{pmatrix}$, with $\boldsymbol{s}_1$ of length w_0 and $\boldsymbol{s}_2$ of length $\ell - w_0$.

Let $\boldsymbol{e}'$ be a solution to the SDP instance $\{\mathbf{H}', \boldsymbol{s}', w'\}$, i.e., $\boldsymbol{e}' \in \mathbb{F}_q^{k+\ell}$ has weight w' and satisfies $\mathbf{H}'\boldsymbol{e}' = \boldsymbol{s}'$. We express $\boldsymbol{e}'$ as the sum of two vectors $\boldsymbol{e}_1 + \boldsymbol{e}_2$, each with length $k+\ell$ and weight w_0.

We then have

$$\mathbf{H}_1(\boldsymbol{e}_1 + \boldsymbol{e}_2) = \boldsymbol{s}_1 \quad \Longrightarrow \quad \mathbf{H}_1\boldsymbol{e}_1 = \boldsymbol{s}_1 - \mathbf{H}_1\boldsymbol{e}_2.$$

We do not know the values of $\mathbf{H}_1\boldsymbol{e}_1$ and $\mathbf{H}_1\boldsymbol{e}_2$. We only know that they sum to $\boldsymbol{s}_1$. We consequently guess that $\mathbf{H}_1\boldsymbol{e}_1$ is equal to some length-w_0 vector $\mathbf{b} \neq \mathbf{0}$, which implies $\mathbf{H}_1\boldsymbol{e}_2 = \boldsymbol{s}_1 - \boldsymbol{b}$.

We find candidates for $\boldsymbol{e}_1$ and $\boldsymbol{e}_2$ using the core idea of the LA-ISD, i.e., by enumerating all subsets of $\{1, \ldots, k+\ell\}$ of size w_0 and then solving the

Algorithm 2: `MitM-LA`

Data: Integers w_0, w', ℓ
Input: Matrix $\mathbf{H} \in \mathbb{F}_q^{r \times n}$, syndrome $\boldsymbol{s} \in \mathbb{F}_q^r$, weight $w \in \mathbb{N}$
Output: Vector $\mathbf{e} \in \mathbb{F}_q^n$ with weight w, such that $\mathbf{He} = \boldsymbol{s}$

1 **while** True **do**
  /* Permute and bring into row reduced echelon form */
2 Sample $\mathbf{P} \xleftarrow{\$} S_n$;
3 If it exists, compute $\mathbf{U} \in GL_r(\mathbb{F}_q)$ so that $\mathbf{UHP} = \begin{pmatrix} \mathbf{H}' & \mathbf{0} \\ \mathbf{H}'' & \mathbf{I}_{n-k-\ell} \end{pmatrix}$, otherwise go back to instruction 2;
4 Set $\mathbf{U}\boldsymbol{s} = \begin{pmatrix} \boldsymbol{s}' \\ \boldsymbol{s}'' \end{pmatrix}$; // $\boldsymbol{s}' \in \mathbb{F}_q^{\ell}$, $\boldsymbol{s}'' \in \mathbb{F}^{r-\ell}$
  /* Split $\mathbf{H}'$ and $\boldsymbol{s}'$, build lists with candidates */
5 Split $\mathbf{H}' = \begin{pmatrix} \mathbf{H}_1 \\ \mathbf{H}_2 \end{pmatrix}$; // $\mathbf{H}_1 \in F_q^{w_0 \times (k+\ell)}$, $\mathbf{H}_2 \in \mathbb{F}_q^{(\ell - w_0) \times (k+\ell)}$
6 Split $\boldsymbol{s}' = \begin{pmatrix} \boldsymbol{s}_1 \\ \boldsymbol{s}_2 \end{pmatrix}$; // $\boldsymbol{s}_1 \in \mathbb{F}_q^{w_0}$, $\boldsymbol{s}_2 \in \mathbb{F}_q^{\ell - w_0}$
7 Initialize $\mathscr{L}_1 = \varnothing$, $\mathscr{L}_2 = \varnothing$;
8 Sample $\boldsymbol{b} \xleftarrow{\$} \mathbb{F}_q^{w_0} \setminus \{\mathbf{0}\}$;
9 **for** $S \subseteq \{1, \dots, k+\ell\}, \quad |S| = w_0$ **do**
10 **if** $\mathrm{rank}\big((\mathbf{H}_1)_S\big) = w_0$ **then**
11 Set $\boldsymbol{e}_1$ so that $(\boldsymbol{e}_1)_S = (\mathbf{H}_1)_S^{-1}\boldsymbol{b}$, and $\boldsymbol{e}_1$ is zero elsewhere;
12 If $\mathrm{wt}(\boldsymbol{e}_1) = w_0$, update $\mathscr{L}_1 \leftarrow \mathscr{L}_1 \cup \{(\boldsymbol{e}_1, \mathbf{H}_2\boldsymbol{e}_1)\}$;
13 Set $\boldsymbol{e}_2$ so that $(\boldsymbol{e}_2)_S = (\mathbf{H}_1)_S^{-1}(\boldsymbol{s}_1 - \boldsymbol{b})$, and $\boldsymbol{e}_2$ is zero elsewhere;
14 If $\mathrm{wt}(\boldsymbol{e}_2) = w_0$, update $\mathscr{L}_2 \leftarrow \mathscr{L}_2 \cup \{(\boldsymbol{e}_2, \boldsymbol{s}_2 - \mathbf{H}_2\boldsymbol{e}_2)\}$;
  /* Find and check collisions */
15 **foreach** $(\boldsymbol{e}_1, \boldsymbol{x}_1) \in \mathscr{L}_1, (\boldsymbol{e}_2, \boldsymbol{x}_2) \in \mathscr{L}_2 : (\boldsymbol{x}_1 = \boldsymbol{x}_2) \wedge \big(\mathrm{wt}(\boldsymbol{e}_1 + \boldsymbol{e}_2) = w'\big)$ **do**
16 Set $\boldsymbol{e}' = \boldsymbol{e}_1 + \boldsymbol{e}_2$;
17 Compute $\boldsymbol{e}'' = \boldsymbol{s}'' - \mathbf{H}''\boldsymbol{e}'$;
18 **if** $\mathrm{wt}(\boldsymbol{e}'') = w - w'$ **then**
19 **return** $\mathbf{P}\begin{pmatrix} \boldsymbol{e}' \\ \boldsymbol{e}'' \end{pmatrix}$;

corresponding linear system. This way, we build two lists $\mathscr{L}_1$, $\mathscr{L}_2$, each of size $L = \binom{k+\ell}{w_0}$ (instructions 7–14).[10]

[10] Notice that requiring $(\mathbf{H}_1)_S$ to be invertible is not necessary, but we do it in order to simplify the description of the algorithm. Indeed, if $(\mathbf{H}_1)_S$, were not invertible, we could still enumerate a solution space. This is just a technicality, but it would have complicated the pseudocode in Algorithm 2. Similarly, we could have allowed for $\boldsymbol{e}_1, \boldsymbol{e}_2$ of weight less than w_0. However, for large q, we do not find such vectors very often, and again it would have complicated the algorithm. Hence, for the sake

The lists $\mathscr{L}_1$ and $\mathscr{L}_2$ are merged (this is implicitly done via the execution of instruction 15), searching for a collision of the form

$$\mathbf{H}_2\boldsymbol{e}_1 = \boldsymbol{s}_2 - \mathbf{H}_2\boldsymbol{e}_2.$$

For each such collision, we get a candidate for $\boldsymbol{e}'$ as $\boldsymbol{e}_1 + \boldsymbol{e}_2$. For each collision, we test if the resulting vector $\boldsymbol{e}'$ has the desired weight w'.

We observe that there are multiple ways to express $\boldsymbol{e}'$ as a sum of two vectors of weight w_0. In fact:

Lemma 1. *Let $\boldsymbol{e}' \in \mathbb{F}_q^{k+\ell}$ be a vector of weight w'. The number of representations of $\boldsymbol{e}'$ as a sum of two vectors $\boldsymbol{e}_1, \boldsymbol{e}_2 \in \mathbb{F}_q^{k+\ell}$ both of weight $w_0 \geq w'/2$ is given by*

$$R = \sum_{j=\max\{0,w'-w_0\}}^{\lfloor w'/2 \rfloor} \binom{w'}{j}\binom{w'-j}{j}(q-2)^{w'-2j}\binom{k+\ell-w'}{w_0-w'+j}(q-1)^{w_0-w'+j}.$$

Proof. Slight reformulation of [22, lemma 7.2.1]. We index our sum with j, which counts the number of coordinates where $\boldsymbol{e}_1$ is zero, while $\boldsymbol{e}_2$ is non-zero. As $\mathrm{wt}(\boldsymbol{e}_1) = \mathrm{wt}(\boldsymbol{e}_2)$, j also counts the number of coordinates where $\boldsymbol{e}_2$ is zero while $\boldsymbol{e}_1$ is non-zero. Otherwise the proof is the same.

The main draw-back of this Meet-in-the-Middle LA-ISD (`MitM-LA`), is the fact that we have to guess $\boldsymbol{b}$. However, the more representations we have, the higher the likelihood that one of them fulfills $\mathbf{H}_1\boldsymbol{e}_1 = \boldsymbol{b} = \boldsymbol{s}_1 - \mathbf{H}_1\boldsymbol{e}_2$. We find the probability of this happening to be

$$1 - (1 - 1/q^{w_0})^R. \tag{7}$$

Before we calculate the runtime of `MitM-LA`, we describe a way to increase the probability above by using the projective idea of [7].

5.2 Solving in the projective space

In Section 3.4 we described a speed-up of Sterns algorithm by [7], where the authors transform an SDP instance into a CFP one via the reduction in Theorem 1. This speed-up is also possible for `MitM-LA`. We take inspiration from the same technique and describe an improvement for `MitM-LA`.

Assume we want to use `MitM-LA` to solve a CFP instance, i.e., search for a weight-w vector $\boldsymbol{e} \in \mathbb{F}_q^n$ such that $\mathbf{H}\boldsymbol{e} = \mathbf{0}$. The first step in `MitM-LA` consists in solving the small instances $\mathbf{H}_1\boldsymbol{e}_1 = \boldsymbol{b}$ and $\mathbf{H}_1\boldsymbol{e}_2 = -\boldsymbol{b}$ for some $\boldsymbol{b} \neq \mathbf{0}$ picked at random. For a fixed solution $\boldsymbol{e}$, the probability that there exists a representation $(\boldsymbol{e}_1, \boldsymbol{e}_2)$ of $\boldsymbol{e}$ fulfilling those equalities is still given by equation (7). But now we can use the fact that if $\boldsymbol{e}$ is a solution to the CFP instance, then $\alpha\boldsymbol{e}$ is also a

of simplicity, we limit ourselves to the case where $(\mathbf{H}_1)_S$ is invertible and $\boldsymbol{e}_1, \boldsymbol{e}_2$ have weight w_0. Note that this limitation implies we can assume $\boldsymbol{b} \neq \mathbf{0}$.

solution for all $\alpha \in \mathbb{F}_q^*$. Hence, we can also look for representations $(\alpha \boldsymbol{e}_1, \alpha \boldsymbol{e}_2)$ of $\alpha \boldsymbol{e}'$. Fixing one representation $(\boldsymbol{e}_1, \boldsymbol{e}_2)$ of $\boldsymbol{e}$, the probability that $\mathbf{H}_1 \alpha \boldsymbol{e}_1 = \boldsymbol{b}$ for some $\alpha \in \mathbb{F}_q^*$ is $(q-1)/q^{w_0}$. And hence the probability that at least one representation of $\alpha \boldsymbol{e}$ for some α solves the smallest instance changes to

$$1 - (1 - (q-1)/q^{w_0})^R.$$

Taking this observation into account, we can derive the time complexity of `MitM-LA` when solving CFP. In particular, we give the time complexity for the "planted" version of the problem.

Proposition 2 (Time complexity of `MitM-LA` for planted CFP). *Let $\mathbf{H} \in \mathbb{F}_q^{r \times n}$ and $w \in \mathbb{N}$ be a random, "planted" instance of CFP. Then, Algorithm 2, on input $\{\mathbf{H}, \mathbf{0}, w\}$, halts in average time $\mathcal{O}\Big(T_{\texttt{MitM-LA}}(q, n, k, w)\Big)$, where*

$$T_{\texttt{MitM-LA}}(q, n, k, w) = \min_{w', \ell, w_0} \left\{ N_{\text{rounds}} \cdot (T_{\text{Gauss}} + T_{\text{lists}} + T_{\text{check}}) \cdot \log_2(q) \right\},$$

with

- $N_{\text{rounds}} = 1/\big(N_w \cdot P_{\text{perm}} \cdot P_{\text{repr}}\big)$, *with* $N_w = 1 + \frac{\binom{n}{w}(q-1)^{w-1}-1}{q^{n-k}}$, $P_{\text{perm}} = \frac{\binom{k+\ell}{w'}\binom{n-k-\ell}{w-w'}}{\binom{n}{w}}$, $P_{\text{repr}} = 1 - (1 - P_{\text{inv}} \cdot (q-1)/q^{w_0})^R$, *$R$ is given in Lemma 1 and*
$$P_{\text{inv}} = 1 - \frac{1}{q} - \frac{1}{q^2}$$
- T_{Gauss} *is as in Eq. (6);*
- $T_{\text{lists}} = w_0^2(w_0+1)\binom{k+\ell}{w_0} + 2(\ell - w_0)w_0 L + 2w_0 L^2/q^{\ell - w_0}$, *with*
$$L = \binom{k+\ell}{w_0}\left(1 - \frac{1}{q} - \frac{1}{q^2}\right)\left(1 - \frac{1}{q}\right)^{w_0};$$
- $T_{\text{check}} = \frac{q}{q-1}(w - w' + 1)w'\left(1 + \frac{q-2}{q-1}\right) f_\ell(w_0, w') \cdot L^2/q^{\ell - w_0}$, *with*

$$f_\ell(w_0, w') = \frac{\sum_i \binom{w_0}{i}\binom{w_0 - i}{2w_0 - 2i - w'}\binom{k+\ell-w_0}{w'+i-w_0}(q-2)^{2w_0-2i-w'}(q-1)^{w'+i-2w_0}}{\binom{k+\ell}{w_0}}.$$

Proof. Let us fix a particular solution $\boldsymbol{e}$ and assume that we have found a permutation $\mathbf{P}$ that permutes exactly w' non-zero entries of $\boldsymbol{e}$ to the first $k + \ell$ coordinates. In this case, we can find $\boldsymbol{e}$ if at least one representation $(\boldsymbol{e}_1, \boldsymbol{e}_2)$ of $\boldsymbol{e}$ satisfies

1. $(\mathbf{H}_1)_{\text{supp}(\boldsymbol{e}_1)}$ and $(\mathbf{H}_1)_{\text{supp}(\boldsymbol{e}_2)}$ are both invertible and
2. $\mathbf{H}_1 \alpha \boldsymbol{e}_1 = \boldsymbol{b} = -\mathbf{H}_1 \alpha \boldsymbol{e}_2$ for some $\alpha \in \mathbb{F}_q^*$

Given a specific representation, the probability of 1. is P_{inv}^2 and the probability of 2. is $(q-1)/q^{w_0}$. Hence the probability of finding $\boldsymbol{e}$ is

$$P_{\text{repr}} = 1 - (1 - P_{\text{inv}}(q-1)/q^{w_0})^R$$

The probability of finding a specific solution $\boldsymbol{e}$ in one iteration is then derived as $P_{\text{perm}}P_{\text{repr}}$, with $P_{\text{perm}} = \frac{\binom{k+\ell}{w'}\binom{n-k-\ell}{w-w'}}{\binom{n}{w}}$ being the probability that $\mathbf{P}$ partitions the weight as desired. Considering also the expected number of solutions N_w, we derive the expected number of rounds.

For the time required to create the lists, we consider that the number of supports we enumerate is $\binom{k+\ell}{w_0}$. For each support, we solve a linear system, with runtime $w_0^2(w_0+1)$. The probability that i) the matrix defining the system is non singular, and ii) the solution has the desired weight w_0 can be taken as $\left(1-1/q-1/q^2\right)\cdot\left(1-\frac{1}{q}\right)^{w_0}$. Hence, we can estimate as $L = \binom{k+\ell}{w_0}\left(1-1/q-1/q^2\right)\cdot\left(1-\frac{1}{q}\right)^{w_0}$ the average size of $\mathscr{L}_1$. Notice that, for each valid $\boldsymbol{e}_1$, we need to compute also $\mathbf{H}_1\boldsymbol{e}_1$: this can be done in time $2(\ell - w_0)w_0$. Note also that the list $\mathscr{L}_2$ does actually not have to be constructed explicitly, as it contains the same elements as $\mathscr{L}_1$, but with a minus.

We can estimate the number of collisions as $N_{\text{cols}} = L^2/q^{\ell-w_0}$. To check if the collision leads to the desired weight, we need to compute $\boldsymbol{e}' = \boldsymbol{e}_1 + \boldsymbol{e}_2$, which takes approximately $2w_0$ sums.

We now proceed to calculate the number of collisions we actually do not discard, i.e., the number of collisions that lead to the desired weight w'. To this end, we derive the probability that each collision $(\boldsymbol{e}_1, \boldsymbol{e}_2)$ produces a vector of the desired weight w'. This can be derived by considering the probability with which the sum of two weight-w_0 vectors each with length $k+\ell$, sampled uniformly at random, yields a vector with weight w'. To this end, let us fix $\boldsymbol{e}_1$ and consider the number of valid vectors $\boldsymbol{e}_2$. Let j denote the number of positions in which both $\boldsymbol{e}_1$ and $\boldsymbol{e}_2$ have non zero entries, so that they sum to a non zero value. Analogously, we denote by i the number of positions in which both $\boldsymbol{e}_1$ and $\boldsymbol{e}_2$ have non zero entries and the sum is zero. To have a resulting weight equal to w', it must be

$$w' = 2(w_0 - j - i) + j \quad \Longrightarrow \quad j = 2w_0 - 2i - w'.$$

See Figure 2 for a visualization of how $\boldsymbol{e}_1$ and $\boldsymbol{e}_2$ should be. The probability that a vector satisfies the above requirements is

$$\begin{aligned} f_\ell(w_0, w') &= \frac{\sum_i \binom{w_0}{i}\binom{w_0-i}{j}\binom{k+\ell-w_0}{w_0-i-j}(q-2)^j(q-1)^{w_0-i-j}}{\binom{k+\ell}{w_0}(q-1)^{w_0}} \\ &= \frac{\sum_i \binom{w_0}{i}\binom{w_0-i}{2w_0-2i-w'}\binom{k+\ell-w_0}{w'+i-w_0}(q-2)^{2w_0-2i-w'}(q-1)^{w'+i-2w_0}}{\binom{k+\ell}{w_0}} \end{aligned}$$

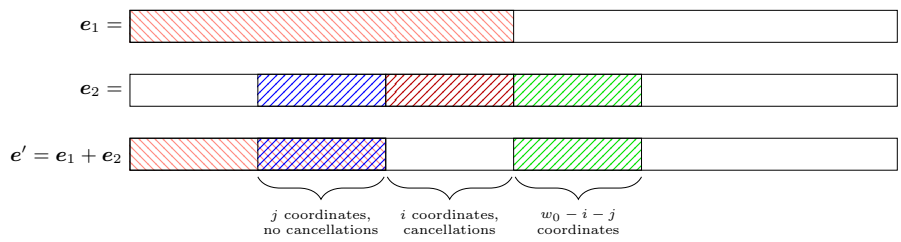


Fig. 2: Visualization of two vector $\boldsymbol{e}_1$ and $\boldsymbol{e}_2$, each with weight w_0, whose sum yields a vector with weight w'. The supports of $\boldsymbol{e}_1$ and $\boldsymbol{e}_2$ overlap in $i + j$ positions: j of these positions yield to a non null value, while the remaining i positions sum to 0.

Hence, the number of vectors we actually check can be estimated as $N_{\text{cols}} \cdot f_\ell(w_0, w')$; and the cost for each check is derived using the early abort strategy of [23, Section 4: Collisions].[11] □

We now derive the complexity of using the above procedure to solve SDP via the reduction to CFP.

Proposition 3 (Time complexity of `MitM-LA` for planted SDP). *Let $\mathbf{H} \in \mathbb{F}_q^{r \times n}$, $\boldsymbol{s} \in \mathbb{F}_q^r$, and $w \in \mathbb{N}$ be a planted SDP instance. Let $\mathbf{G} \in \mathbb{F}_q^{k \times n}$ be a generator matrix for the code having $\mathbf{H}$ as parity-check matrix, and $\widetilde{\mathbf{H}} \in \mathbb{F}_q^{(r-1) \times n}$ be the parity-check matrix of the code whose generator matrix is*

$$\widetilde{\mathbf{G}} = \begin{pmatrix} \mathbf{G} \\ \mathbf{0}\ \boldsymbol{s}^\top \end{pmatrix}.$$

Then, Algorithm 2, on input $\{\widetilde{\mathbf{H}}, \mathbf{0}, w\}$, halts on average in runtime

$$\mathcal{O}\left(T_{\texttt{MitM-LA}}(q, n, k+1, w)\right).$$

The output of the algorithm is a solution to the initial SDP instance with probability $\frac{q-1}{q}$.

6 Numerical results

In this section, we present the runtimes for the most relevant ISD algorithms.[12] We consider instances of schemes from the literature that are based on the non-binary SDP. Specifically, we consider the instances listed in Table 1, which, to the best of our knowledge, are the only instances of schemes based on non-binary SDP.

[11] Explained in a footnote on page 15.
[12] Our code for runtime analysis is available at github.com/FrejaElbro/LAISD.

Source	Instance	(q, n, k, w, d)	Claimed Security
[21]	SDitH_L1_gf256_v11	(256, 242, 126, 87, 1)	143
	SDitH_L1_gf251_v11	(251, 242, 126, 87, 1)	143
	SDitH_L3_gf256_v11	(256, 376, 220, 114, 2)	207
	SDitH_L3_gf251_v11	(251, 376, 220, 114, 2)	207
	SDitH_L5_gf256_v11	(256, 494, 282, 156, 2)	272
	SDitH_L5_gf251_v11	(251, 494, 282, 156, 2)	272
[14]	GPS − 128	(128, 220, 101, 90, 1)	128
	GPS − 256	(256, 207, 93, 90, 1)	128
	GPS − 512	(512, 196, 92, 84, 1)	128
	GPS − 1024	(1024, 187, 90, 80, 1)	128
[8]	CVE	(256, 128, 64, 49, 1)	80

Table 1: Collection of schemes based on non-binary SDP with corresponding parameters and claimed security levels.

Notice that we have an extra parameter, d, which indicates whether one is considering the standard SDP or the d split variant, introduced in [13], in which the sought solution $\boldsymbol{e}$ is split into d blocks with equal length n/d and weight w/d. To get an estimate on the security of this variant of SDP, we rely on [7, Theorem 6.4], which gives a fairly tight lower bound on security.

6.1 Finite regime

First, we compare `LA-ISD` with the other memoryless ISD algorithms, namely, `Prange` and `LeeBrickell`. Notice that, for both `Prange` and `LeeBrickell`, we have employed the natural pivot re-usage strategy (i.e., we used Eq. (6) for the cost of Gaussian elimination). The timings are provided in Table 2.

Notice that, for the instances with $d = 2$, the complexity is a lower bound, which we have computed according to [7, Theorem 6.4].

As we can see from the Table, `LA-ISD` is always the fastest algorithm. Moreover, for many of the considered instances, the algorithm is already capable of bringing the security below the claimed security level.

Forcing the reuse of pivots When computing the values in Table 2, we observed that for some instances, the cost of `Prange` could be reduced by forcing only a few pivots to be reused (the strategy from $[2, 5]$) instead of using the natural pivot reuse strategy from equation 6 and [2]. However, in other instances, the natural pivot reuse strategy performed better. In either case, we found that the resulting runtime was always higher than that of `LeeBrickell` and `LA-ISD`.

In particular, we observe that for the considered instances, `LA-ISD` is not expected to receive a valuable speed-up from this strategy. Indeed, for all the considered instances, the cost of Gaussian elimination is lower than the cost of

	Instance	Prange	LeeBrickell	LA-ISD
$d = 1$	SDitH_L1_gf256_v11	154.3	150.6	**146.3**
	SDitH_L1_gf251_v11	154.3	150.6	**146.3**
	GPS − 128	132.9	128.7	**125.3**
	GPS − 256	131.7	128.4	**124.4**
	GPS − 512	130.3	127.9	**123.2**
	GPS − 1024	128.4	127.0	**121.4**
	CVE	89.7	87.6	**84.1**
$d = 2$	SDitH_L3_gf256_v11	220.4	215.11	**211.0**
	SDitH_L3_gf251_v11	217.87	215.08	**211.0**
	SDitH_L5_gf256_v11	285.09	281.52	**277.4**
	SDitH_L5_gf251_v11	285.09	281.49	**277.4**

Table 2: Comparison between running time of memory-less algorithms, for non binary SDP instances shown in Table 1.

checking candidates for optimal choice of ℓ . Since forcing pivot reuse would not reduce such a cost, but would increase the average number of iterations, this strategy is not convenient for the cases we are considering.

We now consider also the costs of algorithms that use memory. See Table 3 for the a comparison between the running times, taking no cost for accessing memory. For Proj-Stern, we have considered two runtime estimations: one based on the forced pivot reuse strategy from [2,5], as implemented in the code provided by [7], and another based on the natural pivot reuse strategy, as discussed in [2]. The former is computed using the implementation from [7], which is limited to small values of c unless significant computational time is tolerated. The latter, referred to as No CC in the table, corresponds to the natural pivot reuse strategy.

As we see from the Table, our MitM-LA algorithm runs in time which is comparable with that of Proj-Stern. Indeed, the difference between the two algorithms is always very mild and in some cases MitM-LA is (slightly) faster than Proj-Stern.

In particular, our estimates confirm the results in [7], concerning the SDitH signature scheme. Actually, for all instances we provide numbers which are better than those in [7]. The resulting faster attack is either Proj-Stern with natural pivot reuse, or MitM-LA. Finally, for the GPS signature scheme [14], non of the instances meet the desired security level.

We remark that these comparison shall be taken with a grain of salt, since the table essentially shows only estimates of complexities. To provide more insight, one should benchmark implementations of the algorithms, in order to include aspects we are not considering in our analysis, such as the cost of accessing large memory. Still, our results show that for the considered algorithms, security concerns may exist.

	Instance	Stern	Proj-Stern		LA-ISD	MitM-LA
			From [7]	No CC		
$d=1$	SDitH_L1_gf256_v11	146.9	143.2	**142.1**	146.4	142.2
	SDitH_L1_gf251_v11	146.9	143.2	**142.1**	146.3	142.2
	GPS − 128	125.6	122.5	121.4	125.3	**121.3**
	GPS − 256	125.0	121.8	**120.6**	124.4	121.0
	GPS − 512	123.7	120.0	119.9	123.2	**119.4**
	GPS − 1024	122.1	**117.9**	119.2	121.4	118.1
	CVE	84.7	80.9	81.0	84.1	**80.8**
$d=2$	SDitH_L3_gf256_v11	211.0	206.9	206.3	211.0	**206.0**
	SDitH_L3_gf251_v11	210.9	206.9	**206.3**	211.0	206.5
	SDitH_L5_gf256_v11	276.3	272.9	272.5	277.4	**271.4**
	SDitH_L5_gf251_v11	276.2	272.9	272.5	277.4	**272.0**

Table 3: Comparison between running time of several algorithms, for the non binary SDP instances shown in Table 1. No memory cost is applied

Larger values of q We now consider the case of varying q, up to $q \approx 2^{128}$. For such a comparison, we have fixed $n = 256$ and $k = 128$ and, for each value of q, have set the target weight w according to the GV bound. The obtained complexities are shown in Figure 3.

For small values of q, our algorithms are outperformed by `Proj-Stern`. But as q grows, enumeration becomes more expensive and our algorithms become more competitive. For instance, for $q = 2^{128}$, the complexities of `LA-ISD` and `MitM-LA` are, respectively, $2^{249.86}$ and $2^{244.52}$, while `Proj-Stern` runs in time $2^{247.02}$ (`Prange` runs in time $2^{254.50}$).

6.2 Asymptotic analysis of `MitM-LA`

While the speed-up from `MitM-LA` is modest at best for the concrete parameters discussed previously, its advantages become more pronounced in an asymptotic setting for smaller field sizes. We analyze the algorithm's performance by assuming the field size q is a fixed constant while the code length n approaches infinity. For this analysis, it is also standard to assume that the code rate k/n and relative weight w/n converge to constants $\kappa \in [0, 1]$ and $\omega = h_q^{-1}(1 - \kappa)$, respectively. Under these assumptions, polynomial factors in the runtime are dominated by exponential terms, so we ignore them in the complexity analysis. This means that advantages that are only polynomial, such as that of `Proj-Stern` over `Stern`'s algorithm, do not affect the asymptotic exponent. Since Dumer's algorithm is a slight asymptotic improvement on Stern's, we choose `Dumer` for our asymptotic comparison and note that any algorithm that outperforms `Dumer` is also superior to `Proj-Stern` asymptotically.

The performance of an `ISD` algorithm is captured by its *complexity coefficient*, defined as $C_{\texttt{ISD}}(\kappa, q) \triangleq \lim_{n\to\infty} \frac{1}{n} \log_2(T_{\texttt{ISD}})$, where $T_{\texttt{ISD}}$ is the runtime. To compare algorithms, we use the *maximal complexity coefficient*, $C^*_{\texttt{ISD}}(q) \triangleq$

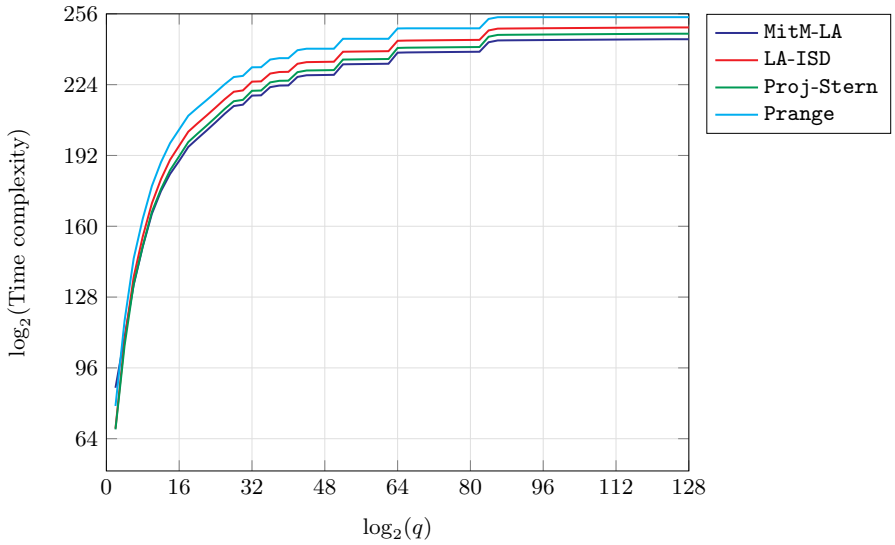


Fig. 3: Time complexities for several ISD algorithms, for $n = 256$, $k = 128$ and several values of q.

$\max_{\kappa \in [0,1]} C_{\texttt{ISD}}(\kappa, q)$. We focus on this worst-case value because it has proven to be a good measure of an algorithm's overall performance, regardless of the specific code rate.

To derive the asymptotic complexity, we begin with the runtime formulas, ignoring polynomial factors. For `MitM-LA`, the runtime is given by:

$$T_{\texttt{MitM-LA}} \in \tilde{\mathcal{O}}\left(P_{\text{perm}}^{-1} \cdot \max(1, R/q^{w_0})^{-1} \cdot \left(L + \frac{L^2}{q^{\ell - w_0}}\right)\right), \quad \text{where } L = \binom{k+\ell}{w_0}.$$

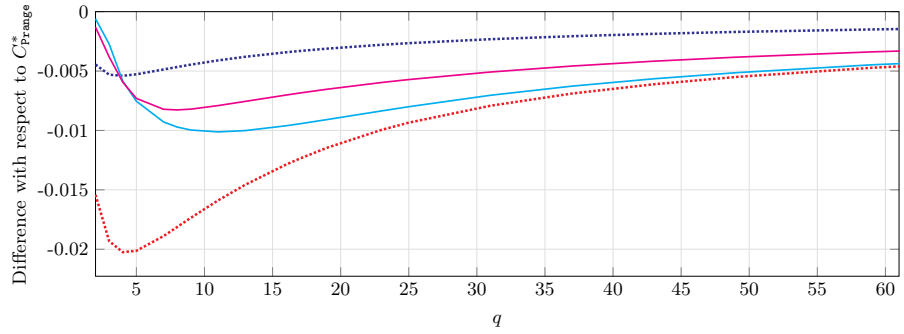
R is the number of representations, which can be approximated by the largest summand in lemma 1. For comparison, the runtime for `Dumer`'s algorithm is:

$$T_{\texttt{Dumer}} \in \tilde{\mathcal{O}}\left(P_{\text{perm}}^{-1} \cdot \left(L + \frac{L^2}{q^{\ell}}\right)\right), \quad \text{where } L = \binom{(k+\ell)/2}{w'/2}(q-1)^{w'/2}.$$

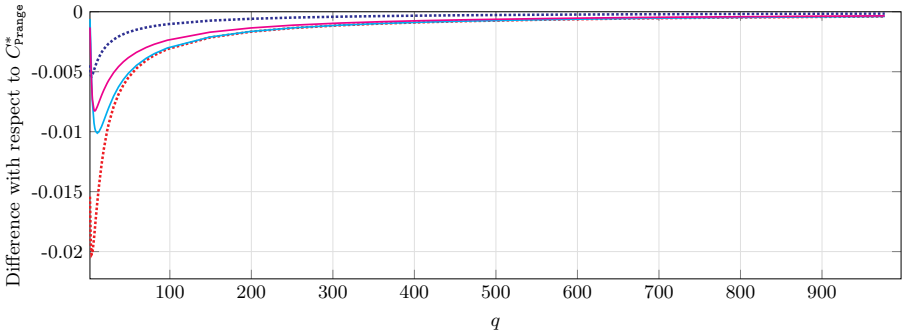
The runtime of `BJMM` without polynomial factors is given in section 3.2.

We approximate the binomial coefficients using the binary entropy function (see equation (1) and assume the internal parameters of the algorithms (e.g., ℓ, w', w_0) grow linearly with n. We then use numerical optimization to find the internal parameter rates that minimize the runtime for each code rate $\kappa \in [0, 1]$, which we sample in steps of 0.01. The maximal complexity coefficient $C^*_{\texttt{ISD}}(q)$ is then the maximum value found across all tested rates. For the parameter sets we analyzed, the two main terms in the runtime complexity (list creation and collision search) are balanced to achieve this optimum.

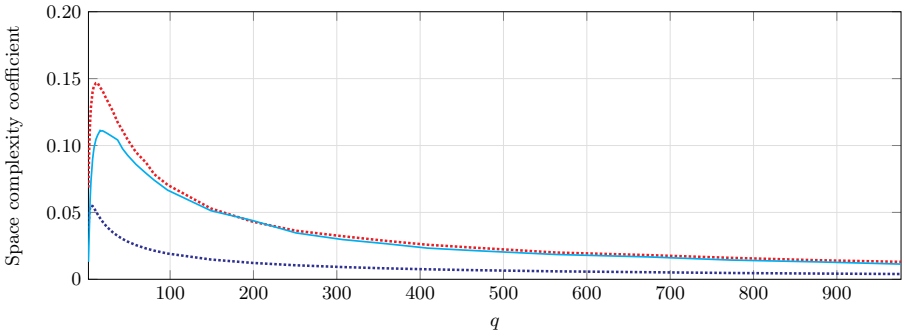
Our results shows that `MitM-LA` is asymptotically faster than `Dumer`'s algorithm for all but the smallest field sizes, while remaining slower than the more



(a) Maximal time complexity coefficient, zoom on small values of q



(b) Maximal time complexity coefficient, showing also large values of q



(c) Maximal space complexity coefficient

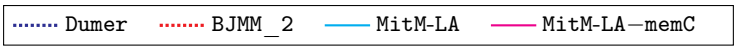


Fig. 4: Figures 4a and 4b show the difference between the worst case time exponents for `Prange` and several other ISD algorithms, as a function of q. Figure 4c reports on the maximal space complexity coefficient as a function of q.

Table 4: Optimization variables for ISD algorithms

	q	C^*	k/n	w'/n	w_c/n	ℓ/n	d/n	i/n	k_r/n
`MitM-LA`	3	0.180	0.45	0.00617	—	0.02001	8.41×10^{-12}	6.55×10^{-19}	—
`-memC`		0.179	0.45	0.00868	2.10×10^{-3}	0.02625	1.82×10^{-5}	6.28×10^{-6}	0.362
Dumer		0.177	0.45	0.01193	—	0.02918	—	—	—
`MitM-LA`	5	0.245	0.45	0.01726	—	0.03625	1.99×10^{-4}	9.52×10^{-5}	—
`-memC`		0.245	0.45	0.01302	0	0.02844	1.47×10^{-4}	6.45×10^{-5}	0.478
Dumer		0.247	0.45	0.01152	—	0.02179	—	—	—
`MitM-LA`	11	0.333	0.45	0.02549	—	0.04162	1.70×10^{-3}	7.64×10^{-4}	—
`-memC`		0.335	0.45	0.01053	0	0.01889	5.60×10^{-4}	2.01×10^{-4}	0.469
Dumer		0.339	0.45	0.00861	—	0.01307	—	—	—
`MitM-LA`	21	0.395	0.45	0.02421	—	0.03676	2.78×10^{-3}	1.30×10^{-3}	—
`-memC`		0.398	0.45	0.00801	2.08×10^{-8}	0.01261	4.14×10^{-4}	1.84×10^{-4}	0.463
Dumer		0.401	0.46	0.00605	—	0.00829	—	—	—
`MitM-LA`	32	0.432	0.45	0.01944	—	0.02862	2.49×10^{-3}	1.21×10^{-3}	—
`-memC`		0.434	0.46	0.00642	0	0.00954	3.25×10^{-4}	1.57×10^{-4}	0.470
Dumer		0.436	0.46	0.00460	—	0.00600	—	—	—
`MitM-LA`	64	0.484	0.46	0.01117	—	0.01541	1.24×10^{-3}	6.68×10^{-4}	—
`-memC`		0.485	0.46	0.00411	0	0.00567	1.96×10^{-4}	1.06×10^{-4}	0.466
Dumer		0.486	0.46	0.00278	—	0.00343	—	—	—

For `MitM-LA-memC`, $w' = w_r + w_c$, so w_c can be calculated as $w' - w_r$.
d is calculated as: `MitM-LA`: $d = w_0 - w'/2$; `MitM-LA-memC`: $d = w_{r0} - w_r/2$.
i is from the formula for representations given in [22, Lemma 7.2.1], and counts half the overlap between $\boldsymbol{e}_1$ and $\boldsymbol{e}_2$.

advanced 2-level `BJMM`. This performance ranking is summarized in Figure 4 and further detailed in Table 4. The data suggests that `MitM-LA`'s advantage over `Dumer` stems from its ability to operate efficiently with a larger optimal relative weight w'/n. Increasing w' raises the probability of a successful permutation, which reduces the number of rounds. The trade-off is that a larger w' makes the lists larger. However, `MitM-LA`'s use of representations mitigates this extra cost, allowing it to leverage a larger w' more effectively than `Dumer` and strike a better balance between the two cost components. This advantage is less pronounced for very small fields, where the term $q-1$ (a key factor in `Dumer`'s cost) is proportionally much smaller than q (a factor in `MitM-LA`'s cost), impacting the trade-off. As q grows, this relative difference becomes negligible.

The core trade-off of our technique, which successfully removes the $(q-1)$ factor from vector enumeration, is the need to guess w_0 entries of the syndrome. This introduces a q^{w_0} cost factor that is apparently only partially offset by representations. This factor is probably the reason that for large q, the optimal w_0 tends to zero, and the performance of `MitM-LA` converges to that of Prange's algorithm, a trait shared by other advanced ISD methods.

The memory complexity of the ISD algorithms is determined by the size of the largest list that must be stored before the final collision search, as the final collisions can be checked on the fly. From figure 4c we see that `MitM-LA`'s memory usage is between that of `Dumer` and `BJMM`. To explore this, we introduce `MitM-LA-memC`, a more flexible variant detailed in the proposition below. This variant introduces parameters that control the support overlap between the vectors $\boldsymbol{e}_1$ and $\boldsymbol{e}_2$; reducing this overlap limits the search space for candidate vectors, thereby shrinking the size of the base lists and the overall memory footprint. We analyze this variant while constraining its memory to be no more than that of `Dumer`'s algorithm. As shown in Figure 4, `MitM-LA-memC` is still faster, demonstrating that our approach is superior for medium sized fields even when operating with the same memory resources. Interestingly, table 4 shows that its optimal strategy under this constraint is to maximize the support overlap, thereby reverting to the original `MitM-LA` approach. This highlights the fundamental efficiency of using representations.

Proposition 4. *Let $\mathbf{H} \in \mathbb{F}_q^{r\times n}$, $\boldsymbol{s} \in \mathbb{F}_q^r$, and $w \in \mathbb{N}$ be a planted SDP instance. There exists an algorithm,* `MitM-LA-memC`*, which solves the SDP on input $(\mathbf{H}, \boldsymbol{s}, w)$ with average running time in*

$$\tilde{\mathcal{O}}\Big(\min_{k_r,k_c,w_c,w_r,w_{r0}} \big\{(P_{\text{repr}}P_{\text{perm}})^{-1}(L+N_{\text{cols}})\big\}\Big)$$

Where

- $P_{\text{perm}} = \frac{\binom{k_r}{w_r}\binom{k_c/2}{w_c/2}^2\binom{n-k_c-k_r}{w-w_c-w_r}}{\binom{n}{w}}$
- $P_{\text{repr}} = \max\{1, R/q^{w_{r0}+w_c/2}\}$ *with R given by*

$$R = \max_i \left\{ \binom{w_r}{i}\binom{w_r-i}{i}(q-2)^{w_r-2i}\binom{k_r-w_r}{w_{r0}-w_r+i}(q-1)^{w_{r0}-w_r+i} \right\}.$$

- $L = \binom{k_r}{w_{r0}}\binom{k_c/2}{w_c/2}$
- $N_{\text{cols}} = L^2/q^{k_c+k_r-k-w_{r0}-w_c/2}$

and where the minimum and maximum are calculated under the restrictions

- $k_r, k_c, w_c, w_r, w_{r0}, i \geq 0$
- $k_r + k_c \leq n$
- $w_c + w_r \leq w$
- $w_c \leq k_c$
- $w_r \leq k_r$
- $w_r/2 \leq w_{r0} \leq k_r$
- $w_r - w_{r0} \leq i \leq w_r/2$
- $w_{r0} - w_r + i \leq k_r - w_r$
- $k_r + k_c - k \geq w_{r0} + w_c/2$

Proof. `MitM-LA-memC` splits $k+\ell$ as $k_c + k_r$, where k_c is the number of positions for which a concatenation merge is performed and k_r is the number of positions that are merged in a way that forms multiple representations. Similarly w' is split as $w_c + w_r$.

The algorithm runs in rounds like all other ISD algorithms. In each round a permutation P is chosen at random. In the round where $\boldsymbol{e}$ is found, $P^{-1}\boldsymbol{e}$ has weight w_r on the first k_r coordinates, $w_c/2$ on the next $k_c/2$ coordinates, $w_c/2$ on following $k_c/2$ coordinates and $w - w_r - w_c$ on the remaining $n - k_c - k_r$ coordinates. The probability of this happening is P_{perm}.

The algorithm behaves exactly the same as `MitM-LA` except for one change. The supports of $\boldsymbol{e}_1$ and $\boldsymbol{e}_2$ are chosen to be only partially overlapping. I.e. We only choose supports S_1 for $\boldsymbol{e}_1$ such that

$$\begin{aligned} S_1 \cap \{1, .., k_r\} &= w_{r0} \\ S_1 \cap \{k_r + 1, .., k_r + k_c/2\} &= w_c/2 \\ S_1 \cap \{k_r + k_c/2 + 1, .., k_r + k_c\} &= 0 \end{aligned}$$

Similarly, supports S_2 for $\boldsymbol{e}_2$ are chosen such that

$$\begin{aligned} S_2 \cap \{1, .., k_r\} &= w_{r0} \\ S_2 \cap \{k_r + 1, .., k_r + k_c/2\} &= 0 \\ S_2 \cap \{k_r + k_c/2 + 1, .., k_r + k_c\} &= w_c/2 \end{aligned}$$

This means the baselists have size L (defined in the proposition) and we estimate the number of collisions to be N_{cols} (also defined in the proposition).

The number of representations $(\boldsymbol{e}_1, \boldsymbol{e}_2)$ of $\boldsymbol{e}'$ is given by lemma 1 with $k+\ell$ replaced by k_r, w' replaced by w_r and w_0 replaced by w_{r0}. Denoting the number of representations by R, we estimate the probability of at least one representation $(\boldsymbol{e}_1, \boldsymbol{e}_2)$ of $\boldsymbol{e}'$ fulfilling $\mathbf{H}_1\boldsymbol{e}_1 = \boldsymbol{b} = \boldsymbol{s}_1 - \mathbf{H}_1\boldsymbol{e}_2$ to be P_{repr}.

Up to polynomial factors, the number of q-ary operations per round is given by $L + N_{\text{cols}}$. A round is successful if it both distributes the weight of $\boldsymbol{e}$ correctly and chooses a successful $\boldsymbol{b}$. Putting all this together gives the runtime formula in the proposition. □

7 Conclusions

In this paper, we have proposed two new ISD algorithms, `LA-ISD` and `MitM-LA`, for solving non-binary SDP. Both algorithms are based on a common principle, that is, avoiding the full enumeration of all low weight vectors by exploiting linear algebra relations. Namely, in both algorithms, we numerate only the supports and recover non-zero values by solving a (small) linear system. This idea is at the base of `LA-ISD` and gets enhanced in `MitM-LA` by employing a meet-in-the-middle approach and representations.

For both algorithms, we provide a precise analysis for the finite regime and provide costs for the instances of schemes that appeared in the literature. `LA-ISD`

results in the fastest memory-less algorithm since it outperforms significantly both `Prange` and `LeeBrickell`. For what concerns `MitM-LA`, we show that is competitive with the state-of-the-art algorithm provided in [7] and, for many parameters sets, turns out to be slightly faster. Our findings show that all instances recommended in [14] do not meet the claimed security level. And concerning the SDitH signature scheme [21], we confirm that the instances recommended in Version 1.1 fall short of security.

In the asymptotic regime, we show that `MitM-LA` falls between `Dumer` and `BJMM` for moderate values value of q while, for growing q, the performance of all the ISD-algorithms converge to that of `Prange`. Hence, the existence of algorithms that can outperform `Prange` in this regime is left as an open question.

References

1. Becker, A., Joux, A., May, A., Meurer, A.: Decoding random binary linear codes in $2^{n/20}$: How 1+1 = 0 improves information set decoding. In: Advances in Cryptology - EUROCRYPT 2012. LNCS, Springer (2012)
2. Bernstein, D.J., Lange, T., Peters, C.: Attacking and defending the McEliece cryptosystem. In: Post-Quantum Cryptography 2008. LNCS, vol. 5299, pp. 31–46 (2008)
3. Both, L., May, A.: Optimizing BJMM with Nearest Neighbors: Full Decoding in $2^{2/21n}$ and McEliece Security. In: WCC Workshop on Coding and Cryptography (Sep 2017)
4. Both, L., May, A.: Decoding linear codes with high error rate and its impact for lpn security. In: International Conference on Post-Quantum Cryptography. pp. 25–46. Springer (2018)
5. Canteaut, A., Chabaud, F.: A new algorithm for finding minimum-weight words in a linear code: Application to McEliece's cryptosystem and to narrow-sense BCH codes of length 511. IEEE Transactions on Information Theory **44**(1), 367–378 (1998)
6. Carrier, K., Debris-Alazard, T., Meyer-Hilfiger, C., Tillich, J.P.: Reduction from sparse LPN to LPN, dual attack 3.0. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 286–315. Springer (2024)
7. Carrier, K., Hatey, V., Tillich, J.P.: Projective space Stern decoding and application to SDitH. In: International Conference on Applied Cryptography and Network Security. pp. 29–52. Springer (2024)
8. Cayrel, P.L., Véron, P., El Yousfi Alaoui, S.M.: A zero-knowledge identification scheme based on the q-ary syndrome decoding problem. In: International Workshop on Selected Areas in Cryptography. pp. 171–186. Springer (2010)
9. Ducas, L., Esser, A., Etinski, S., Kirshanova, E.: Asymptotics and improvements of sieving for codes. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 151–180. Springer (2024)
10. Dumer, I.: On minimum distance decoding of linear codes. In: Proc. 5th Joint Soviet-Swedish Int. Workshop Inform. Theory. pp. 50–52. Moscow (1991)
11. Esser, A., Bellini, E.: Syndrome decoding estimator. In: IACR International Conference on Public-Key Cryptography. pp. 112–141. Springer (2022)

12. Esser, A., Verbel, J.A., Zweydinger, F., Bellini, E.: SoK: CryptographicEstimators - a Software Library for Cryptographic Hardness Estimation. In: AsiaCCS. ACM (2024)
13. Feneuil, T., Joux, A., Rivain, M.: Syndrome decoding in the head: Shorter signatures from zero-knowledge proofs. In: Annual International Cryptology Conference. pp. 541–572. Springer (2022)
14. Gueron, S., Persichetti, E., Santini, P.: Designing a practical code-based signature scheme from zero-knowledge proofs with trusted setup. Cryptography **6**(1), 5 (2022)
15. Guo, Q., Johansson, T., Nguyen, V.: A new sieving-style information-set decoding algorithm. IEEE Transactions on Information Theory (2024)
16. Howgrave-Graham, N., Joux, A.: New generic algorithms for hard knapsacks. In: Gilbert, H. (ed.) Advances in Cryptology - EUROCRYPT. LNCS, vol. 6110. Sringer (2010)
17. Lee, P.J., Brickell, E.F.: An observation on the security of McEliece's public-key cryptosystem. In: Advances in Cryptology - EUROCRYPT'88. LNCS, vol. 330, pp. 275–280. Springer (1988)
18. Leon, J.: A probabilistic algorithm for computing minimum weights of large error-correcting codes. IEEE Trans. Inform. Theory **34**(5), 1354–1359 (1988)
19. May, A., Meurer, A., Thomae, E.: Decoding random linear codes in $O(2^{0.054n})$. In: Lee, D.H., Wang, X. (eds.) Advances in Cryptology - ASIACRYPT 2011. LNCS, vol. 7073, pp. 107–124. Springer (2011)
20. May, A., Ozerov, I.: On computing nearest neighbors with applications to decoding of binary linear codes. In: Oswald, E., Fischlin, M. (eds.) Advances in Cryptology - EUROCRYPT 2015. LNCS, vol. 9056, pp. 203–228. Springer (2015)
21. Melchor, C., Gueron, S., Feneuil, T., Howe, J., Persichetti, E., Joseph, D., Gama, N., Joux, A., Randrianarisoa, T., Rivain, M., Yue, D.: The syndrome decoding in the head (SD-in-the-Head) signature scheme. Second round submission to the NIST post-quantum cryptography call (Feb 2025), `https://sdith.org/`
22. Meurer, A.: A Coding-Theoretic Approach to Cryptanalysis. Ph.D. thesis, Ruhr University Bochum (Nov 2017), `http://www.cits.rub.de/imperia/md/content/diss.pdf`
23. Peters, C.: Information-set decoding for linear codes over $\mathbf{F}_q$. In: Post-Quantum Cryptography 2010. LNCS, vol. 6061, pp. 81–94. Springer (2010)
24. Prange, E.: The use of information sets in decoding cyclic codes. IRE Transactions on Information Theory **8**(5), 5–9 (1962). `https://doi.org/10.1109/TIT.1962.1057777`, `http://dx.doi.org/10.1109/TIT.1962.1057777`
25. Stern, J.: A method for finding codewords of small weight. In: Cohen, G.D., Wolfmann, J. (eds.) Coding Theory and Applications. LNCS, vol. 388, pp. 106–113. Springer (1988)

A Factorizing Gaussian Elimination

We describe in the appendix a variant of *Factorizing Gaussian Elimination* introduced by [7], inspired by the earlier works of [23] and [2]. The key observation is that a single Gaussian elimination step (which is relatively expensive) can be reused to test multiple zero-windows. As a bonus, one can also test several different supports for $\boldsymbol{e}_1$ and $\boldsymbol{e}_2$. The following is a description of Stern's algorithm with Factorizing of the Gaussian Elimination step. But clearly the idea of Factorizing Gaussian Elimination can be applied to many more algorithms.

1. Choose an information set for $\mathbf{H}$, $J \subset \{1, \ldots, n\}$ of size k.
2. Set $\mathbf{P} \in S_n$ so that the coordinates indexed by J are moved to the leftmost k coordinates and, with Gaussian Elimination compute $\mathbf{U} \in GL_{n-k}(\mathbb{F}_q)$ such that
$$\mathbf{UHP} = \left(\mathbf{A}\ \mathbf{I}_{n-k}\right),$$
with $\mathbf{A}$ having size $(n-k) \times k$.
3. Repeat the following N_{in} times, where N_{in} is a algorithm parameter chosen to minimise runtime
   (a) Choose a set $K \subseteq \{1, \ldots, n-k\}$ of size ℓ randomly. And denote by $\mathbf{H}'$ the rows of $\mathbf{A}$ indexed by K and $\mathbf{H}''$ the rows of $\mathbf{A}$ indexed by K^C. Similarly, let $\boldsymbol{s}' = (\mathbf{U}\boldsymbol{s})_K$ and $\boldsymbol{s}'' = (\mathbf{U}\boldsymbol{s})_{K^C}$
   (b) Choose a random partition $J_1 \cup J_2 = \{1, \ldots, k\}$ with $|J_1| = |J_2| = k/2$ and build lists of candidate $\boldsymbol{e}_1$'s and $\boldsymbol{e}_2$'s and follows
$$\mathcal{L}_1 = \{(\boldsymbol{e}_1, \mathbf{H}'\boldsymbol{e}_1) \mid \boldsymbol{e}_1 \in \mathbb{F}_q^k, \mathrm{supp}(\boldsymbol{e}_1) \subseteq J_1, \mathrm{wt}(\boldsymbol{e}_1) = w'/2\}$$
$$\mathcal{L}_2 = \{(\boldsymbol{e}_2, \boldsymbol{s}' - \mathbf{H}'\boldsymbol{e}_2) \mid \boldsymbol{e}_2 \in \mathbb{F}_q^k, \mathrm{supp}(\boldsymbol{e}_2) \subseteq J_2, \mathrm{wt}(\boldsymbol{e}_2) = w'/2\}.$$
   (c) Find solutions $\boldsymbol{e}'$ to $\mathbf{H}'\boldsymbol{e}' = \boldsymbol{s}'$ via a meet-in-the-middle collision search between $\mathcal{L}_1$ and $\mathcal{L}_2$. And for each solution, check the weight of $\boldsymbol{e}'' = \boldsymbol{s}'' - \mathbf{H}''\boldsymbol{e}'$.
   (d) If an $\boldsymbol{e}''$ is found such that $\mathrm{wt}(e'') = w - w'$, return $\mathbf{P}\begin{pmatrix}\boldsymbol{e}' \\ \boldsymbol{e}''\end{pmatrix}$.
4. If no solution is returned in the above loop. Go back to step 1 and try a new information set.

The probability of success can now be factorized. Given a specific solution $\boldsymbol{e}$, the probability that the outer loop finds J such that $\boldsymbol{e}_J$ has weight w', is
$$P_{out} = \frac{\binom{k}{w'}\binom{n-k}{w-w'}}{\binom{n}{w}}.$$

Assuming $\boldsymbol{e}_J$ has weight w', the probability that one iteration of the inner loop succeeds in finding $\boldsymbol{e}_J$ is
$$q_{in} = \frac{\binom{(k+\ell)/2}{w'/2}^2}{\binom{k+\ell}{w'}} \cdot \frac{\binom{n-k-\ell}{w-w'}}{\binom{n-k}{w-w'}}.$$

Hence, the probability that $\boldsymbol{e}_J$ is found in at least one of the N_{in} iterations of the inner loop is
$$P_{in} = 1 - (1 - q_{in})^{N_{in}}$$
and the runtime needed to find one specific solution $\boldsymbol{e}$ to the syndrome decoding problem is in
$$\mathcal{O}(P_{out}^{-1} \cdot (T_{Gauss} + N_{in} \cdot (T_{lists} + T_{check} \cdot N_{cols}))),$$
where $T_{Gauss}, T_{lists}, T_{check}, N_{cols}$ have the same meaning as for Dumer's algorithm.

B Details on Natural Reuse of Pivots

The size of $J^C \cap \{k+1, \ldots, n\}$ follows a hypergeometric distribution, then hence its average value is

$$u = \sum_{x=\max(0,n-2k)}^{n-k} x \cdot \frac{\binom{n-k}{x}\binom{k}{n-k-x}}{\binom{n}{n-k}} = \frac{(n-k)^2}{n}. \tag{8}$$

For each row $i \in \{1, .., n-k-u\}$ (starting from row $n-k-u$ and working upwards to row 1) we have to

1. scale the row, and
2. subtract a scaled version of the row from all the other rows

The first step requires $k+i-1$ q-ary operations, since all entries to the right of position $k+i$ are zero. Furthermore, scaling position $k+i$ will always result in 1, so no calculation is needed. The second takes time $2(n-k-1)(k+i-1)$ for similar reasons.

The total number of operations is thus

$$T_{\text{Gauss}} = \sum_{i=1}^{n-k-u} k+i-1+2(n-k-1)(k+i-1) = (n-k-\tfrac{1}{2})(n-k-u)(n+k-u-1)$$